

IEEE P1619™/D16

블록 지향 저장장치 데이터의 암호학적 보호를 위한 표준

Standard for Cryptographic Protection of Data on
Block-Oriented Storage Devices

Security in Storage Working Group
IEEE Computer Society Committee

한국어 학습 번역본

원문: IEEE P1619/D16, May 2007

학습 주석 | 이 문서는 승인 전 Draft입니다. 원문 자체가 “unapproved IEEE Standards Draft, subject to change”라고 명시하므로 규격 적합성 판단에는 사용하면 안 됩니다. 여기서는 XTS-AES 구조를 학습하고 구현을 대조하기 위한 목적으로 번역했습니다.

번역 원칙

AES, XTS-AES, tweak, data unit, Key1, Key2, ciphertext stealing, $GF(2^{128})$, LBA, pseudocode 등의 기술 명칭은 구현 문서·코드와 직접 대응하도록 English notation을 유지합니다. English 기술어가 남는 경우 바로 옆 설명 또는 학습 주석에 한국어 뜻을 제공합니다. 수식, 기호, hexadecimal 값, Test Vector는 가능한 한 원문 형식을 유지합니다.

Abstract - 초록

이 표준은 sector-level storage device의 데이터를 보호하기 위한 cryptographic transform과 key archival 방법을 규정한다.

Keywords: Security(보안), Storage(저장장치), Encryption(암호화), Key Management(키 관리).

학습 주석 | sector-level storage는 SSD/HDD처럼 데이터를 sector 또는 logical block 단위로 읽고 쓰는 저장장치를 의미합니다. 이 문서의 핵심은 “저장 위치를 tweak에 포함하여 각 위치의 128-bit block을 서로 다른 것처럼 암호화”하는 XTS-AES입니다.

Introduction - 소개

(이 Introduction은 IEEE P1619/D16 표준 본문 규정의 일부가 아닌 설명 자료이다.)

이 표준의 목적은 공격자가 저장된 데이터에 접근할 수 있는 위협 모델에서, sector-based device에 저장되는 데이터를 암호화하는 방법을 설명하는 것이다. 서로 다른 구현 사이의 호환성을 위해 encryption transform과 encryption key의 export/import 방법을 규정한다. data in transit, 즉 전송 중인 데이터의 암호화는 이 표준의 범위에 포함되지 않는다.

이 표준은 XTS-AES tweakable block cipher와 sector-based storage encryption에 이를 사용하는 방법을 정의한다. XTS-AES는 128 bit 이상인 data unit에 작용하며 AES block cipher를 내부 subroutine으로 사용한다. XTS-AES의 key material은 data encryption key와 tweak key로 구성된다. tweak key는 암호화할 data block의 논리적 위치를 암호 연산에 포함시키는 데 사용된다.

XTS-AES는 [XEX04]의 tweakable block cipher class를 구체적으로 구현한 방식이다. XTS-AES는 copy-and-paste 공격과 dictionary attack 같은 위협을 완화하면서, 구현에서 parallelization과 pipelining을 허용한다.

Patents - 특허

이 표준을 구현할 때 patent right가 적용되는 기술이 필요할 가능성이 있다. 이 표준의 발행은 특정 patent의 존재 또는 유효성에 대해 IEEE가 입장을 표명하는 것이 아니다. IEEE는 표준 구현에 license가 필요한 patent 또는 patent application을 식별하거나, 제기된 patent의 법적 유효성 또는 범위를 조사할 책임이 없다.

Participants - 참여자

Security in Storage Working Group은 다음 sponsor 체계에서 운영되었다.

Sponsor: John L. Cole, Information Assurance Standards Committee

Co-Sponsor: Curtis Anderson, Storage Systems Standards Committee

초기 Working Group officers: James P. Hughes (Chair), Serge Plotkin (Vice-chair), Fabio Maino (Secretary).

Draft 완료 시점 officers: Matthew V. Ball (Chair), Eric Hibbard (Vice-chair), Fabio Maino (Secretary).

P1619 Task Group Membership - 구성원

Draft 완료 시점 P1619 Task Group의 구성원은 다음과 같다.

구성원	구성원
Serge Plotkin – Task Group Chair and Technical Editor	Gideon Avida
Matthew V. Ball	David L. Black
Russel Dietz	Rob Elliott
Hal Finney	John Geldman
Robert Griffin	Cyril Guyot
Shai Halevi	Laszlo Hars
Eric A. Hibbard	Larry D. Hofer
Walt Hubis	James P. Hughes
Glen Jaquette	Curt Kolovson
Robert A. Lockhart	Fabio Maino
Charlie Martin	David A. McGrew
Dalit Naor	Landon Curt Noll
Jim Norton	Scott Painter
David Sheehy	Robert N. Snively
Douglas L. Whiting	

이 문서는 Serge Plotkin, Shai Halevi, Dalit Naor가 편집했다.

Special thanks: Doug Whiting, Brian Gladman, Rob Elliott.

Balloting committee 명단은 당시 Draft에서 “to be supplied by IEEE”로 남아 있다.

CONTENTS - 목차

1. Overview	1
1.1 Scope	1
1.2 Purpose	1
1.3 Related work	1
2. References	2
3. Definitions, acronyms and abbreviations	2
3.1 Definitions	2
3.2 Acronyms and abbreviations	2
4. Special Terms	3
4.1 Numerical values	3
4.2 Letter symbols	3
5. The XTS-AES transform	3
5.1 Data units and tweaks	3
5.2 Multiplication by a primitive element α	4
5.3 The XTS-AES Encryption Procedure	4
5.4 The XTS-AES Decryption Procedure	6
6. Using XTS-AES-128 and XTS-AES-256 for Encryption of Storage	8
7. Exporting and Archiving XTS-AES-128 and XTS-AES-256 keys	9
7.1 Key Backup Structure	9
7.2 XML Format	11
7.3 Encryption of Key Backup material	13

CONTENTS - 목차 (계속)

Annex A (informative) Bibliography	15
Annex B (informative) Test Vectors	16
Annex C (informative) Pseudocode for XTS-AES-128 and XTS-AES-256 Encryption	25
C.1 Encryption of a data unit with a size that is a multiple of 16 bytes	25
C.2 Encryption of a data unit with a size that is not a multiple of 16 bytes	26
Annex D (informative) Rationale and Design Choices	27
D.1 Purpose	27
D.2 Transparent Encryption	27
D.3 Wide vs. Narrow Block Tweakable Encryption	28
D.4 The XEX Construction	29
D.5 Sector-size that is not a multiple of 128 bits	32
D.6 Miscellaneous	32

Standard for Cryptographic Protection of Data on Block-Oriented Storage Devices

1. Overview - 개요

1.1 Scope - 범위

이 표준은 block-oriented storage device의 데이터를 cryptographic하게 보호하기 위한 architecture의 요소를 규정하며, 사용할 data protection method, algorithm, mode를 설명한다.

1.2 Purpose - 목적

이 표준은 일정한 길이의 block으로 저장되는 데이터를 cryptographic하게 보호하는 architecture의 구체적 요소를 정의한다. 이런 mechanism을 표준화하면 저장 데이터 보호를 더 안전하고 상호운용 가능한 방식으로 구현할 수 있다.

여기서 정의하는 XTS-AES transform은 공격자가 stored data에 접근할 가능성이 있고, 보호할 데이터가 고정 크기 unit(sector, logical disk block 등)으로 제공되며, 각 unit을 다른 data unit과 독립적으로 처리해야 하는 storage encryption을 위해 설계되었다.

XTS-AES는 length-preserving transform이다. 즉 XTS-AES가 생성하는 ciphertext 길이는 plaintext 길이와 같다. 또한 각 data unit을 독립적으로 처리할 수 있다. 이 두 특성 때문에 기존 구성요소의 data layout을 바꾸지 않고 encryption/decryption module을 삽입하는 transparent encryption이 가능하다.

학습 주석 | transparent encryption의 대가는 security limitation입니다. ciphertext에 authentication tag를 추가할 수 없고, 서로 다른 sector 사이에 chaining을 할 수 없습니다. Annex D가 이 설계 trade-off를 상세히 설명합니다.

표준은 XTS-AES transform 자체의 encryption/decryption 절차와 stored data에 적용하는 방법을 포함한다. 또한 key-export format을 규정한다. 목표는 한 표준 적합 장치가 XTS-AES로 암호화한 데이터를 다른 표준 적합 장치가 export된 key를 import하여 복호화할 수 있도록 하는 것이다.

1.3 Related work - 관련 연구

Tweakable block cipher의 security goal에 대한 형식적 정의는 Liskov, Rivest, Wagner [LRW02]에서 제시되었고, standard block cipher로 tweakable cipher를 구성하는 방법도 설명되었다. Schroepel [S98]은 tweak을 native하게 포함하는 cipher를 설계하여 이 아이디어를 더 이른 시기에 제안했다.

2. References - 참조 문서

이 문서의 적용에 필수적인 referenced document는 다음과 같다. 날짜가 지정된 reference는 해당 edition만 적용하고, 날짜가 없는 reference는 amendment/corrigenda를 포함한 최신 edition을 적용한다.

NIST FIPS-197, Federal Information Processing Standard (FIPS) for the Advanced Encryption Standard (AES).

REC-xml, W3C Extensible Markup Language (XML) 1.0 (Fourth Edition).

3. Definitions, acronyms and abbreviations

3.1 Definitions - 정의

key scope: 특정 key로 암호화되는 데이터의 범위. 동일한 크기의 data unit으로 나뉜다. key scope는 첫 data unit에 대응하는 tweak value, data unit size, data length를 나타내는 세 개의 non-negative integer로 식별된다.

tweak value: XTS-AES로 암호화 또는 복호화하는 데이터의 logical position을 나타내는 128-bit value.

data unit: IEEE Std 1619에서 key scope 내부의 128 bit 이상 데이터. 첫 data unit은 key scope의 첫 bit에서 시작하며 이후 unit은 바로 이전 unit의 끝 다음 bit에서 시작한다. 같은 key scope 안의 data unit 크기는 동일하다. data unit은 storage device의 physical block과 반드시 일치할 필요가 없다.

3.2 Acronyms and abbreviations

약어	English	한국어 의미
AES	Advanced Encryption Standard	고급 암호화 표준
DTD	Document Type Definition	XML 문서 형식 정의
FIPS	Federal Information Processing Standard	미 연방 정보처리 표준
GF	Galois Field	갈루아 체 / 유한체
LBA	Logical Block Address	논리 블록 주소
XML	Extensible Markup Language	확장 가능한 마크업 언어
XTS	XEX encryption mode with tweak and ciphertext stealing	tweak와 ciphertext stealing을 사용하는 XEX 기반 mode
Base64	Base64 encoding	binary data를 문자로 표현하는 encoding

4. Special Terms - 특수 표기

4.1 Numerical values - 수 표현

이 문서에서는 decimal number와 binary number를 사용한다. 일반적으로 count는 decimal로, bit pattern은 binary로 표현한다. Decimal은 0, 1, 2, ...의 일반 표기이고 binary는 bit string 뒤에 subscript 2를 붙인다. 예: decimal 26 = 00011010₂.

4.2 Letter symbols - 기호

기호	의미
$\oplus$	bit-wise exclusive-OR (XOR)
$\otimes$	GF(2) polynomial을 $x^{128} + x^7 + x^2 + x + 1$ 로 modulo reduction하는 multiplication
α	GF(2 ¹²⁸)의 primitive element, polynomial x에 대응
$\leftarrow$	assignment
	concatenation, 예: K1=001 ₂ , K2=101010 ₂ 이면 K1 K2=001101010 ₂
//	comment 시작. 줄 끝까지 comment
$\lfloor x \rfloor$	floor, 예: $\lfloor 7/3 \rfloor = 2$

5. The XTS-AES transform

5.1 Data units and tweaks

이 표준은 encryption 대상 data stream을 연속된 동일 크기 data unit으로 나누어 처리한다. 여기서 data stream은 storage device에 저장하기 위해 암호화해야 하는 정보만을 의미하며, 암호화하지 않는 정보는 data stream 바깥으로 본다.

data unit size는 최소 128 bit이어야 한다. 한 data unit의 128-bit block 수는 $2^{128}-2$ 를 넘으면 안 되며, 권고상 2^{20} 을 넘지 않는 것이 좋다. 각 data unit에는 non-negative integer tweak value가 하나씩 부여되고, 임의의 non-negative integer에서 시작해 연속적으로 증가한다.

AES로 tweak value를 암호화하기 전에는 이를 little-endian byte array로 바꾼다. 예를 들어 tweak value 123456789A₁₆는 byte array 9a₁₆, 78₁₆, 56₁₆, 34₁₆, 12₁₆에 대응한다.

학습 주석 | little-endian은 낮은 significance의 byte를 먼저 배치한다는 뜻입니다. 저장장치에서 LBA를 128-bit tweak 입력으로 만들 때 byte ordering을 잘못 처리하면 모든 XTS 결과가 틀어집니다.

data unit과 storage device상의 transfer, placement, composition 사이의 mapping은 이 표준의 범위 밖이다. 표준 적합 장치는 이 mapping을 문서화하는 것이 좋다. 특히 하나의 data unit이 반드시 하나의 logical block과 일치할 필요는 없다. 여러 logical block을 하나의 data unit으로 둘 수도 있고, logical block 일부만 암호화하는 경우에는 암호화되는 byte만 data stream에 포함된다.

5.2 Multiplication by a primitive element α

뒤의 encryption/decryption procedure는 AES encryption 또는 decryption 결과인 16-byte value에 $GF(2^{128})$ 의 primitive element α 의 j 번째 power를 곱한다. 입력 16-byte value는 byte array $a_0[k]$, $k=0,1,\dots,15$ 로 본다. $a_0[0]$ 은 AES block의 첫 번째 byte이다.

입력과 출력은 다음과 같다.

Input: $j = \alpha$ 의 power

byte array $a_0[k]$, $k=0,1,\dots,15$

Output: byte array $a_j[k]$, $k=0,1,\dots,15$

i 를 0부터 j 까지 반복하면서 output array를 다음 recurrence로 정의한다.

$$a_{i+1}[0] \leftarrow (2(a_i[0] \bmod 128)) \oplus (135 \square a_i[15]/128 \square)$$

$$a_{i+1}[k] \leftarrow (2(a_i[k] \bmod 128)) \oplus \square a_i[k-1]/128 \square, \quad k=1,2,\dots,15$$

학습 주석 | 개념적으로는 16 byte 전체를 1 bit left shift하면서 carry를 다음 byte로 전달합니다. 마지막 15번째 byte에서 carry가 발생하면 첫 byte에 decimal 135 = 0x87을 XOR합니다. 이 0x87은 $GF(2^{128})$ modulus polynomial $x^{128}+x^7+x^2+x+1$ 에서 나옵니다.

5.3 The XTS-AES Encryption Procedure

5.3.1 XTS-AES-blockEnc Procedure - 단일 128-bit block 암호화

단일 128-bit block에 대한 XTS-AES encryption은 다음으로 모델링한다.

$$C \leftarrow \text{XTS-AES-blockEnc}(\text{Key}, P, i, j)$$

Key: 256 또는 512 bit XTS-AES key

P: 128-bit plaintext block

i: 128-bit tweak value

j: data unit 내부 128-bit block의 sequential number

C: 결과 128-bit ciphertext block

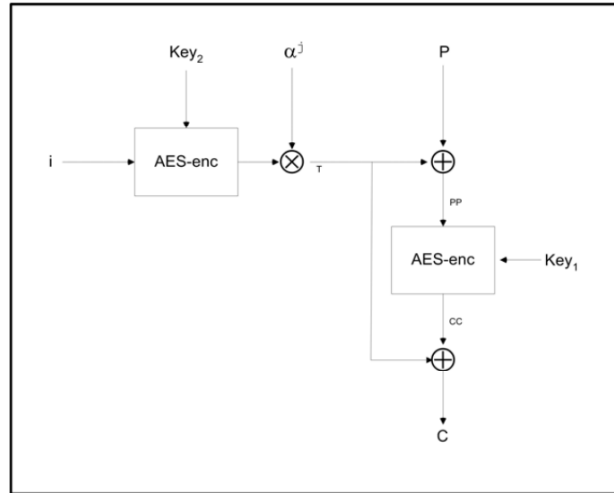
Key는 같은 크기의 두 field Key1과 Key2로 나눈다: $\text{Key} = \text{Key1} \mid \text{Key2}$.

Ciphertext는 다음 단계 또는 그와 동등한 절차로 계산한다.

1. $T \leftarrow \text{AES-enc}(\text{Key}_2, i) \otimes \alpha^j$
2. $PP \leftarrow P \oplus T$
3. $CC \leftarrow \text{AES-enc}(\text{Key}_1, PP)$
4. $C \leftarrow CC \oplus T$

$\text{AES-enc}(K, P)$ 는 FIPS-197에 따라 key K로 plaintext P를 AES encryption하는 절차이다. Step 1의 multiplication과 power 계산은 $\text{GF}(2^{128})$ 에서 수행하며 α 는 4.2의 primitive element이다.

Figure 1 - XTS-AES blockEnc procedure



5.3.2 XTS-AES Encryption of a data unit

128 bit 이상인 plaintext data unit P에 대한 XTS-AES encryption은 다음으로 모델링한다.

$$C \leftarrow \text{XTS-AES-Enc}(\text{Key}, P, i)$$

Key: 256 또는 512 bit XTS-AES key

P: plaintext

i: 128-bit tweak value

C: P와 동일한 bit-size의 ciphertext

plaintext data unit은 $m+1$ 개의 block으로 나눈다:

$$P = P_0 \parallel \dots \parallel P_{m-1} \parallel P_m$$

m 은 128m이 P의 bit-size를 넘지 않는 가장 큰 정수이다. 처음 m 개 block $P_0 \dots P_{m-1}$ 은 각각 정확히 128 bit이고, 마지막 P_m 은 0~127 bit이다. P_m 은 empty일 수 있다. Key는 Key1||Key2로 나눈다.

Ciphertext C는 다음 또는 동등한 절차로 계산한다.

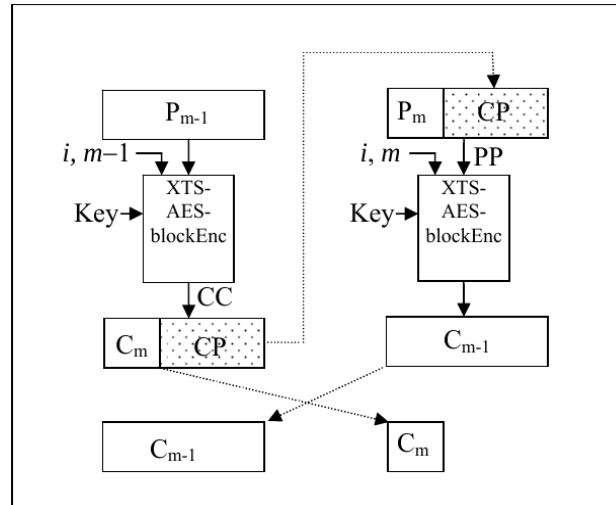
```

1. for q ← 0 to m-2 do
  Cq ← XTS-AES-blockEnc(Key, Pq, i, q)
2. b ← bit-size of Pm
3. if b = 0 then
  Cm-1 ← XTS-AES-blockEnc(Key, Pm-1, i, m-1)
  Cm ← empty
4. else
  CC ← XTS-AES-blockEnc(Key, Pm-1, i, m-1)
  Cm ← first b bits of CC
  CP ← last (128-b) bits of CC
  PP ← Pm | CP
  Cm-1 ← XTS-AES-blockEnc(Key, PP, i, m)
5. C ← C0 || ... || Cm-1 || Cm

```

학습 주석 | 마지막 block이 128 bit보다 짧을 때 사용하는 절차가 ciphertext stealing (CTS)입니다. padding으로 길이를 늘리지 않고, 마지막 full block에서 필요한 ciphertext bit를 "빌려" 전체 ciphertext 길이를 plaintext와 동일하게 유지합니다.

Figure 2 - 마지막 block이 1~127 bit일 때 마지막 두 block의 XTS-AES encryption



5.4 The XTS-AES Decryption Procedure

5.4.1 XTS-AES-blockDec Procedure - 단일 128-bit block 복호화

단일 128-bit block의 XTS-AES decryption은 다음으로 모델링한다.

$$P \leftarrow \text{XTS-AES-blockDec}(\text{Key}, C, i, j)$$

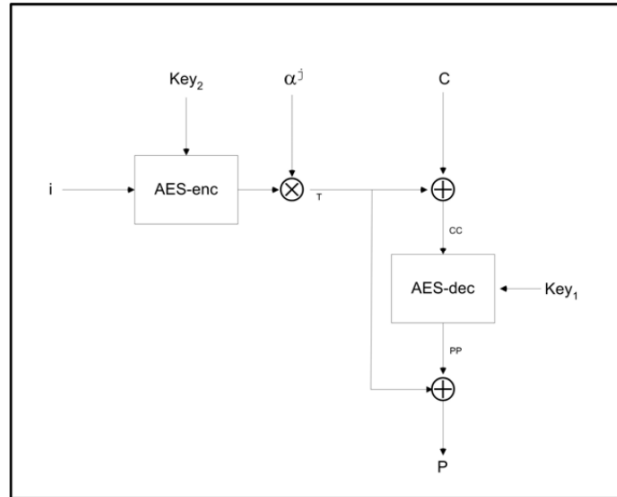
Key: 256 또는 512-bit XTS-AES key, C: 128-bit ciphertext block, i: 128-bit tweak, j: data unit 내부 block number, P: 결과 plaintext block.

Key는 Key1과 Key2로 나누며 $\text{Key} = \text{Key1} \parallel \text{Key2}$ 이다. Plaintext는 다음 또는 동등한 절차로 계산한다.

1. $T \leftarrow \text{AES-enc}(\text{Key2}, i) \otimes \alpha^j$
2. $CC \leftarrow C \oplus T$
3. $PP \leftarrow \text{AES-dec}(\text{Key1}, CC)$
4. $P \leftarrow PP \oplus T$

$\text{AES-dec}(K, C)$ 는 FIPS-197에 따라 key K로 ciphertext C를 AES decryption하는 절차이다. Step 1의 multiplication과 power 계산은 $\text{GF}(2^{128})$ 에서 수행한다.

Figure 3 - XTS-AES blockDec procedure



5.4.2 XTS-AES Decryption of a Data Unit

128 bit 이상인 data unit ciphertext C의 XTS-AES decryption은 다음으로 모델링한다.

$$P \leftarrow \text{XTS-AES-Dec}(\text{Key}, C, i)$$

Key: 256/512-bit XTS-AES key, C: data unit ciphertext, i: 128-bit tweak, P: C와 같은 bit-size의 plaintext.

Ciphertext는 $m+1$ 개의 block으로 분할한다:

$$C = C_0 \parallel \dots \parallel C_{m-1} \parallel C_m$$

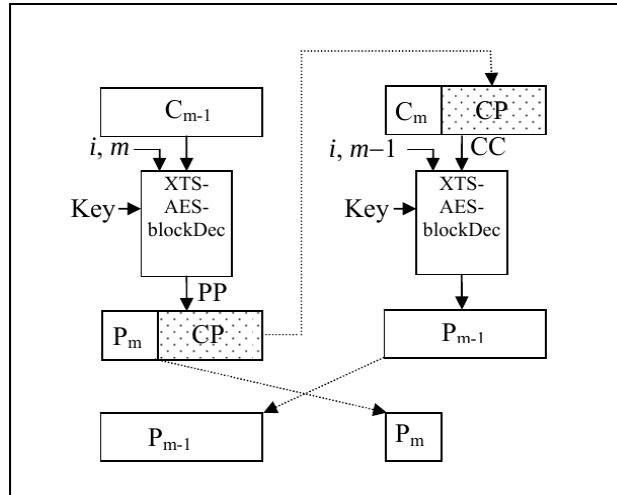
m 은 128m이 C 의 bit-size를 넘지 않는 가장 큰 정수이고, $C_0 \dots C_{m-1}$ 은 128 bit, C_m 은 0~127 bit이다. $Key = Key1 \parallel Key2$ 로 나눈다.

```

1. for  $q \leftarrow 0$  to  $m-2$  do
   $Pq \leftarrow XTS\text{-}AES\text{-}blockDec(Key, Cq, i, q)$ 
2.  $b \leftarrow \text{bit-size of } C_m$ 
3. if  $b = 0$  then
   $P_{m-1} \leftarrow XTS\text{-}AES\text{-}blockDec(Key, C_{m-1}, i, m-1)$ 
   $P_m \leftarrow \text{empty}$ 
4. else
   $PP \leftarrow XTS\text{-}AES\text{-}blockDec(Key, C_{m-1}, i, m)$ 
   $P_m \leftarrow \text{first } b \text{ bits of } PP$ 
   $CP \leftarrow \text{last } (128-b) \text{ bits of } PP$ 
   $CC \leftarrow C_m \parallel CP$ 
   $P_{m-1} \leftarrow XTS\text{-}AES\text{-}blockDec(Key, CC, i, m-1)$ 
5.  $P \leftarrow P_0 \parallel \dots \parallel P_{m-1} \parallel P_m$ 

```

Figure 4 - 마지막 block이 1~127 bit일 때 마지막 두 block의 XTS-AES decryption



6. Using XTS-AES-128 and XTS-AES-256 for Encryption of Storage

5.3.2와 5.4.2의 encryption/decryption procedure는 AES를 기본 building block으로 사용한다. 전체 XTS-AES key가 256 bit이면 내부 AES는 128-bit key를 사용하며 이를 XTS-AES-128이라 한다. 전체 XTS-AES key가 512 bit이면 내부 AES는 256-bit key를 사용하며 이를 XTS-AES-256이라 한다. 표준 적합 구현은 두 mode 중 적어도 하나를 지원해야 한다.

학습 주석 | 중요: XTS-AES-128의 전체 key는 128 bit가 아니라 256 bit입니다. Key1=128 bit + Key2=128 bit이기 때문입니다. XTS-AES-256의 전체 key는 512 bit입니다.

Key scope는 하나의 XTS-AES key로 암호화되는 데이터 범위를 정의한다. 7.1.4와 같이 세 정수로 표현한다: (1) 첫 data unit에 연결된 tweak value, (2) 각 data unit의 bit 크기, (3) 이 key로 encryption/decryption할 data unit의 개수. 표준 적합 구현은 여러 data unit size를 지원할 수도 있고 하나만 지원할 수도 있다.

Disk의 sector-level encryption에 적용하면 data unit은 일반적으로 logical block과 대응하고, key scope는 연속된 logical block 범위와 대응하며, key scope의 첫 data unit tweak value는 일반적으로 첫 logical block의 LBA(Logical Block Address)와 대응한다.

하나의 XTS-AES key는 둘 이상의 key scope와 연결하면 안 된다. 같은 key와 같은 index를 여러 block에 재사용하면 security vulnerability가 생길 수 있으며, 특히 trivial cut-and-paste attack을 가능하게 한다.

7. Exporting and Archiving XTS-AES-128 and XTS-AES-256 keys

7.1 Key Backup Structure

7.1.1 Key Backup Structure Overview

표준 적합 장치를 둘러싼 system은 이 clause의 Key Backup structure를 지원하는 것이 좋다. XML 표현은 7.2에 제시한다. Key Backup structure는 XTS-AES-[128,256]으로 암호화한 연속된 data unit을 복호화하는 데 필요한 모든 정보를 제공한다.

첫 data unit에는 KeyScopeStart element의 tweak value가 연결된다. 예를 들어 하나의 physical medium에서 특정 XTS-AES key로 암호화되는 첫 data unit의 위치를 initial tweak value로 둘 수 있다.

학습 주석 | initial tweak value를 backup 구조에 넣는 이유는 여러 key를 한 media에서 쓰거나 일부 range만 archive할 때도 정확한 위치 tweak를 복원하기 위해서입니다. tweak 시작값을 모르면 동일한 Key1/Key2가 있어도 원래 ciphertext를 올바르게 복호화할 수 없습니다.

Table 1은 Key Backup structure의 element를 나열한다.

Element	Description - 한국어	Reference
StructureID	현재 backup structure instance를 식별	Table 2
Standard	사용한 표준 식별자	Table 3
KeyScope	key가 적용되는 data 범위	Table 4
Transform	사용한 transform 설명	Table 5
KeyMaterial	key material과 길이	Table 7

7.1.2 Structure ID

StructureID는 특정 Key Backup structure instance를 고유하게 식별하는 정보를 담는다.

Element	Size	XML Encoding	Description - 한국어
ID	16 bytes	Base64	Key Backup structure의 일반 식별자
Comment	Up to 1024 bytes	Text	vendor가 제공하는 설명 text

7.1.3 Standard

Standard element는 data unit을 암호화할 때 사용한 표준 정보를 담는다.

Element	Size	XML Encoding	Description - 한국어
StandardNumber	Up to 128 bytes	Text	사용 표준 번호, IEEE STD 1619-2007
StandardComment	Up to 256 bytes	Text	추가 standard 관련 정보

7.1.4 Key Scope

KeyScope는 Key Backup structure에서 식별되는 key material의 적용 범위를 나타낸다. 특정 위치부터 연속 번호가 부여된 ordered data unit sequence이다.

Table 4 – KeyScope

Element	Size	XML Encoding	Description - 한국어
KeyScopeStart	16 bytes	Integer	scope의 첫 data unit에 대응하는 tweak value
DataUnitSize	16 bytes	Integer	현재 key가 적용되는 한 data unit의 bit 수
KeyScopeLength	16 bytes	Integer	현재 key가 적용되는 data unit 개수

7.1.5 Transform

Element	Size	XML Encoding	Description
TransformName	Up to 16 bytes	Text	transform 이름

TransformName은 다음 중 하나여야 한다.

String	Description - 한국어
XTS-AES-128	이 표준에 정의된 XTS-AES-128 transform
XTS-AES-256	이 표준에 정의된 XTS-AES-256 transform

7.1.6 Key Material

KeyMaterial element의 모든 key material은 secret이다.

Element	Size	XML Encoding	Description - 한국어
KeyLength	2 bytes	Integer	key 길이(bit). XTS-AES-128은 256, XTS-AES-256은 512
KeyValue	variable	Base64	key 실제 값

7.2 XML Format

Key Backup structure는 통일된 형식과 application-independent한 key 공유를 위해 XML로 encoding한다. 이를 통해 Key Backup structure를 자동 생성하고 parsing할 수 있다. Key Backup Format의 DTD는 Figure 5에 제시된다.

학습 주석 | XML/DTD/Base64는 XTS 암호 수학 자체가 아니라 key backup/export 형식을 표준화하기 위한 부분입니다. FPGA datapath 구현 학습에서는 Section 5와 Annex C가 우선순위가 더 높습니다.

Figure 5 - Key Backup format용 DTD

```
<!ELEMENT KeyBackup (StructureID, Standard, KeyScope, Transform, KeyMaterial)>
<!ELEMENT StructureID (ID, Comment?)>
<!ELEMENT ID (#PCDATA)>
<!ATTLIST ID Encoding CDATA #FIXED "Base64">
<!ELEMENT Comment (#PCDATA)>
<!ELEMENT Standard (StandardNumber, StandardComment?)>
<!ELEMENT StandardNumber (#PCDATA)>
<!ELEMENT StandardComment (#PCDATA)>
<!ELEMENT KeyScope (KeyScopeStart, DataUnitSize, KeyScopeLength)>
<!ELEMENT KeyScopeStart (#PCDATA)>
<!ATTLIST KeyScopeStart Encoding CDATA #FIXED "Integer">
<!ELEMENT DataUnitSize (#PCDATA)>
<!ATTLIST DataUnitSize Encoding CDATA #FIXED "Integer">
<!ELEMENT KeyScopeLength (#PCDATA)>
<!ATTLIST KeyScopeLength Encoding CDATA #FIXED "Integer">
<!ELEMENT Transform (TransformName)>
<!ELEMENT TransformName (#PCDATA)>
<!ELEMENT KeyMaterial (KeyLength, KeyValue)>
<!ELEMENT KeyLength (#PCDATA)>
<!ATTLIST KeyLength Encoding CDATA #FIXED "Integer">
<!ELEMENT KeyValue (#PCDATA)>
<!ATTLIST KeyValue Encoding CDATA #FIXED "Base64">
```

Figure 6 - 하나의 key scope를 담는 XML document example

```
<?xml version="1.0" encoding="ISO-8859-1" standalone="no"?>
<!DOCTYPE KeyBackup SYSTEM "keybackup.dtd">
<KeyBackup>
  <StructureID>
    <ID Encoding="Base64">YUB1JHJqMDNhWjFAJCVwXQ==</ID>
    <Comment>Comment text here</Comment>
  </StructureID>
  <Standard>
    <StandardNumber>IEEE STD 1619-2007</StandardNumber>
    <StandardComment>Disk</StandardComment>
  </Standard>
  <KeyScope>
    <KeyScopeStart Encoding="Integer">0</KeyScopeStart>
    <DataUnitSize Encoding="Integer">4096</DataUnitSize>
    <KeyScopeLength Encoding="Integer">1083</KeyScopeLength>
  </KeyScope>
  <Transform><TransformName>XTS-AES-256</TransformName></Transform>
  <KeyMaterial>
    <KeyLength Encoding="Integer">512</KeyLength>
    <KeyValue Encoding="Base64">... Base64 key value ...</KeyValue>
  </KeyMaterial>
</KeyBackup>
```

학습 주석 | DTD/XML tag와 Base64 값은 machine-readable field이므로 번역하지 않고 원문 표기를 유지했습니다. 의미는 7.1의 표에 한국어로 정리되어 있습니다.

7.3 Encryption of Key Backup material

Key Backup structure는 다음과 같이 보호할 수 있다. 실제 key material(Table 7의 KeyMaterial)은 xml-enc [XML-ENC]를 사용해 wrap/encrypt하여 XML key backup structure 안에 embed해야 한다.

[XML-ENC]는 단일 key wrapping algorithm을 강제하지 않지만, 이 표준에 conform하는 implementation은 NIST AES CBC 256을 지원해야 한다. [XML-ENC]가 허용하는 다른 key wrap algorithm도 사용할 수 있다.

key element를 wrapping하는 데 사용하는 key, 즉 KEK(Key Encryption Key)는 [XML-KMS]로 reference할 수 있다. wrapping key의 저장 위치는 이 표준이 규정하지 않는다. wrapping key의 cryptographic strength는 wrapping 대상 storage encryption key와 적어도 동등해야 한다.

예를 들어 XTS-AES-512 key를 wrap하려면 AES-256 key를 사용하는 것이 좋다. implementation은 standard method를 이용해 key file의 integrity도 제공하는 것이 좋다.

Figure 7의 example에서는 KeyMaterial을 AES-256 Key Wrap으로 암호화하며 wrapping key는 KeyInfo element의 identifier “WrapKey”로 reference한다. 이 XML example은 Figure 6의 unwrapped example과 대응한다.

Example wrapping key의 Base64 encoding:

```
9s7VKn6PYK0X+Yis50ERn0CDA3MmEd5+TnYnZv+PVVrn=
```

학습 주석 | KEK는 storage data를 직접 암호화하는 XTS key와 별개로, “XTS key 자체를 안전하게 보관하기 위해” 사용하는 key입니다. 실제 제품에서는 key hierarchy/KMS/HSM 설계와 연결됩니다.

Figure 7 - Figure 6의 Key Scope에서 KeyMaterial을 AES-256 방식으로 wrapping한 XML

```

<KeyBackup>
  <StructureID>
    <ID Encoding="Base64">YUB1JHJqMDNhWjFAJCVwXQ==</ID>
    <Comment>Comment text here</Comment>
  </StructureID>
  <Standard>
    <StandardNumber>IEEE STD 1619-2007</StandardNumber>
    <StandardComment>Disk</StandardComment>
  </Standard>
  <KeyScope>
    <KeyScopeStart Encoding="Integer">0</KeyScopeStart>
    <DataUnitSize Encoding="Integer">4096</DataUnitSize>
    <KeyScopeLength Encoding="Integer">1083</KeyScopeLength>
  </KeyScope>
  <Transform><TransformName>XTS-AES-256</TransformName></Transform>
  <KeyMaterial>
    <KeyLength Encoding="Integer">512</KeyLength>
    <KeyValue Encoding="Base64">
      <xenc:EncryptedData xmlns:xenc="http://www.w3.org/2001/04/xmlenc#"
        Type="http://www.w3.org/2001/04/xmlenc#Content">
        <xenc:EncryptionMethod Algorithm="http://www.w3.org/2001/04/xmlenc#aes256-cbc"/>
        <ds:KeyInfo xmlns:ds="http://www.w3.org/2000/09/xmldsig#">
          <ds:KeyName>WrapKey</ds:KeyName>
        </ds:KeyInfo>
        <xenc:CipherData><xenc:CipherValue>
          M1uzVD5PGeoneuFP0bgG3o1bzGVRrCLG5pR00ER/eTyBTRNSEdm0yT3Q/2
          Z6dNn4plzIAm15QYgCKj0TJMPWxzZFZH75/S3SHAha0Yhy4DXovhf+LiiXvThqxWcGaIXS6a4+X82vBgT8j2JRqPe/+A==
        </xenc:CipherValue></xenc:CipherData>
      </xenc:EncryptedData>
    </KeyValue>
  </KeyMaterial>
</KeyBackup>

```

학습 주석 | XML namespace, tag name, Algorithm URI, Base64 ciphertext는 실제 interchange format이므로 원문 표기를 그대로 유지했습니다.

Annex A (informative) - Bibliography / 참고문헌

참고문헌의 title, author, publication 명칭은 원문과의 대조를 위해 English를 유지한다.

- [MM82] C.H. Meyer and S. M. Matyas, "Cryptography: a New Dimension in Computer Data Security." John Wiley & Sons, 1982.
- [NR98] M. Naor and O. Reingold. "A pseudo-random encryption mode." Manuscript.
- [LRW02] M. Liskov, R. Rivest, and D. Wagner. "Tweakable block ciphers." CRYPTO 2002.
- [XEX04] P. Rogaway. "Efficient Instantiations of Tweakable Blockciphers and Refinements to Modes OCB and PMAC". Asiacrypt 2004.
- [S98] R. Schroepel. "The Hasty Pudding cipher." The first AES conference, NIST, 1998.
- [HR03] S. Halevi and P. Rogaway. "A tweakable enciphering mode." CRYPTO 2003.
- [HR04] S. Halevi and P. Rogaway. "A parallelizable enciphering mode." RSA-CT 2004.
- [H04] S. Halevi. "EME*: extending EME to handle arbitrary-length messages with associated data," INDOCRYPT 2004.
- [KEY-MGMT] NIST Key Management Guidelines SP800-57.
- [XML-ENC] XML Encryption Syntax and Processing. W3C.
- [XML-KMS] XML Key Management Specification (XKMS). W3C.
- [Crypto-HBook] A. Menezes, P. Oorshot and S. Vanstone. "Handbook of Applied Cryptography," CRC Press, 1997.
- [RFC3548] The Base16, Base32, and Base64 Data Encodings. IETF, July 2003.

2 (informative)

t(평문)
 ext(암호문)
 mask
 XTS key
 uence
 a unit 순번
 r 값은
 문 유지

- PTX prefix denotes plaintext;
- CTX prefix denotes ciphertext.
- TWK prefix denotes the mask computed in step 1 in 5.3.1.

```
Key1 00000000000000000000000000000000
Key2 00000000000000000000000000000000
Data Unit Sequence number 0
PTX 0000000000000000000000000000000000000000000000000000000000000000
TWK 66e94bd4ef8a2c3b884cfa59ca342beccd297a8df1559761099f4b39469565c
CTX 917cf69ebd68b2ec9b9fe9a3eadda692cd43d2f59598ed858c02c2652fbf922e
```

[illegible][illegible]

```
Key1 27182818284590452353602874713526
Key2 31415926535897932384626433832795
Data Unit Sequence number 0

PTX 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
PTX 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
PTX 404142434445464748494a4b4c4d4e4f505152535455565758595a5b5c5d5e5f
PTX 606162636465666768696a6b6c6d6e6f707172737475767778797a7b7c7d7e7f
PTX 808182838485868788898a8b8c8d8e8f909192939495969798999a9b9c9d9e9f
PTX a0a1a2a3a4a5a6a7a8a9aaabacadaeafb0b1b2b3b4b5b6b7b8b9babbcbcbdbefb
PTX c0c1c2c3c4c5c6c7c8c9cacbcccdcecf0d1d2d3d4d5d6d7d8d9dadbdcddeedf
PTX e0e1e2e3e4e5e6e7e8e9eaebecedeeeff0f1f2f3f4f5f6f7f8f9fafbfcfdfef
PTX 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
PTX 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
```

1 PTX 404142434445464748494a4b4c4d4e4f505152535455565758595a5b5c5d5e5f
 2 PTX 606162636465666768696a6b6c6d6e6f707172737475767778797a7b7c7d7e7f
 3 PTX 808182838485868788898a8b8c8d8e8f909192939495969798999a9b9c9d9e9f
 4 PTX a0a1a2a3a4a5a6a7a8a9aaabacadaeafb0b1b2b3b4b5b6b7b8b9babbbcbdbbfbf
 5 c0c1c2c3c4c5c6c7c8c9cacbcccdcecfcd0d1d2d3d4d5d6d7d8d9dadbdcddeeff
 6 e0e1e2e3e4e5e6e7e8e9eaebeceedeefeff0f1f2f3f4f5f6f7f8f9fafbfcfdfeff
 7 27a7479bfa1d476489f308cd4cfa6e2a96e4bbe3208ff25287dd3819616e89c
 8 c78cf7f5e543445f8333d8fa7f56000005279fa5d8b5e4ad40e736ddb4d35412
 9 328063fd2aab53e5ea1e0a9f332500a5df9487d07a5c92cc512c8866c7e860ce
 10 93fdf166a24912b422976146ae20ce846bb7dc9ba94a767aaef20c0d61ad0265
 11 5ea92dc4c4e41a8952c651d33174be51a10c421110e6d81588ede82103a252d8
 12 a750e8768defffed9122810aaeb99f9172af82b604dc4b8e51bcb08235a6f434
 13 1332e4ca60482a4ba1a03b3e65008fc5da76b70bf1690db4eae29c5f1badd03c
 14 5ccf2a55d705ddcd86d449511ceb7ec30bf12b1fa35b913f9f747a8afd1b130e
 15 94bff94effd01a91735ca1726acd0b197c4e5b03393697e126826fb6bbde8ecc
 16 CTX 1e08298516e2c9ed03ff3c1b7860f6de76d4cecd94c8119855ef5297ca67e9f3
 17 CTX e7ff72b1e99785ca0a7e7720c5b36dc6d72cac9574c8cbbc2f801e23e56fd344
 18 CTX b07f22154beba0f08ce8891e643ed995c94d9a69c9f1b5f499027a78572aeebd
 19 CTX 74d20cc39881c213ee770b1010e4bea718846977ae119f7a023ab58cca0ad752
 20 CTX afe656bb3c17256a9f6e9bf19fdd5a38fc82bbe872c5539edb609ef4f79c203e
 21 CTX bb140f2e583cb2ad15b4aa5b655016a8449277dbd477ef2c8d6c017db738b18d
 22 CTX eb4a427d1923ce3ff262735779a418f20a282df920147beabe421ee5319d0568
 23
 24 Vector 5
 25 Key1 27182818284590452353602874713526
 26 Key2 31415926535897932384626433832795
 27 Data Unit Sequence Number 01
 28
 29 PTX 27a7479bfa1d476489f308cd4cfa6e2a96e4bbe3208ff25287dd3819616e89c
 30 PTX c78cf7f5e543445f8333d8fa7f56000005279fa5d8b5e4ad40e736ddb4d35412
 31 PTX 328063fd2aab53e5ea1e0a9f332500a5df9487d07a5c92cc512c8866c7e860ce
 32 PTX 93fdf166a24912b422976146ae20ce846bb7dc9ba94a767aaef20c0d61ad0265
 33 PTX 5ea92dc4c4e41a8952c651d33174be51a10c421110e6d81588ede82103a252d8
 34 PTX a750e8768defffed9122810aaeb99f9172af82b604dc4b8e51bcb08235a6f434
 35 PTX 1332e4ca60482a4ba1a03b3e65008fc5da76b70bf1690db4eae29c5f1badd03c
 36 PTX 5ccf2a55d705ddcd86d449511ceb7ec30bf12b1fa35b913f9f747a8afd1b130e
 37 PTX 94bff94effd01a91735ca1726acd0b197c4e5b03393697e126826fb6bbde8ecc
 38 PTX 1e08298516e2c9ed03ff3c1b7860f6de76d4cecd94c8119855ef5297ca67e9f3
 39 PTX e7ff72b1e99785ca0a7e7720c5b36dc6d72cac9574c8cbbc2f801e23e56fd344
 40 PTX b07f22154beba0f08ce8891e643ed995c94d9a69c9f1b5f499027a78572aeebd
 41 PTX 74d20cc39881c213ee770b1010e4bea718846977ae119f7a023ab58cca0ad752
 42 PTX afe656bb3c17256a9f6e9bf19fdd5a38fc82bbe872c5539edb609ef4f79c203e
 43 PTX bb140f2e583cb2ad15b4aa5b655016a8449277dbd477ef2c8d6c017db738b18d
 44 PTX eb4a427d1923ce3ff262735779a418f20a282df920147beabe421ee5319d0568
 45 CTX 264d3ca8512194fec312c8c9891f279fefdd608d0c027b60483a3fa811d65ee5
 46 CTX 9d52d9e40ec5672d81532b38b6b089ce951f0f9c35590b8b978d175213f329bb
 47 CTX 1c2fd30f2f7f30492a61a532a79f51d36f5e31a7c9a12c286082ff7d2394d18f
 48 CTX 783e1a8e72c722caaaa52d8f065657d2631fd25bfd8e5baad6e527d763517501
 49 CTX c68c5edc3cdd55435c532d7125c8614deed9adaa3acade5888b87bef641c4c99
 50 CTX 4c8091b5bcd38f73963fb5bc37aa922fbfe3df4e5b915e6eb514717bdd2a7407
 51 CTX 9a5073f5c4bdf46adf7d282e7a393a52579d11a028da4d9cd9c77124f9648ee3
 52 CTX 83b1ac763930e7162a8d37f350b2f74b8472cf09902063c6b32e8c2d9290cefb
 53 CTX d7346d1c779a0df50edcde4531da07b099c638e83a755944df2aef1aa31752fd
 54 CTX 323dcb710fb4bfb9d22b925bc3577e1b8949e729a90bbafecf7f7879e7b114
 55 CTX 7e28ba0bae940db795a61b15ecf4df8db07b824bb062802cc98a9545bb2aaeed
 56 CTX 77cb3fc6db15dcd7d80d7d5bc406c4970a3478ada8899b329198eb61c193fb62
 57 CTX 75aa8ca340344a75a862aeb92eee1ce032fd950b47d7704a3876923b4ad6284
 58 CTX 4bf4a09c4dbe8b4397184b7471360c9564880aeddb9baa4af2e75394b08cd32
 59 CTX ff479c57a07d3eab5d54de5f9738b8d27f27a9f0ab11799d7b7ffefb2704c95c
 60 CTX 6ad12c39f1e867a4b7b1d7818a4b753dfd2a89ccb45e001a03a867b187f225dd
 61
 62 Vector 6
 63 Key1 27182818284590452353602874713526
 64 Key2 31415926535897932384626433832795

1 Data Unit Sequence Number 02

2

3 PTX 264d3ca8512194fec312c8c9891f279fefdd608d0c027b60483a3fa811d65ee5

4 PTX 9d52d9e40ec5672d81532b38b6b089ce951f0f9c35590b8b978d175213f329bb
 1c2fd30f2f7f30492a61a532a79f51d36f5e31a7c9a12c286082ff7d2394d18f
 783e1a8e72c722caaaa52d8f065657d2631fd25bfd8e5baad6e527d763517501
 c68c5edc3cdd55435c532d7125c8614deed9adaa3acade5888b87bef641c4c99
 4c8091b5bcd387f3963fb5bc37aa922fbfe3df4e5b915e6eb514717bdd2a7407
 9a5073f5c4bfd46adf7d282e7a393a52579d11a028da4d9cd9c77124f9648ee3
 83b1ac763930e7162a8d37f350b2f74b8472cf09902063c6b32e8c2d9290cefb
 d7346d1c779a0df50edcde4531da07b099c638e83a755944df2aef1aa31752fd
 323dcb710fb4bfbb9d22b925bc3577e1b8949e729a90bbafecaf7f7879e7b114
 7e28ba0bae940db795a61b15ecf4df8db07b824bb062802cc98a9545bb2aaeed
 77cb3fc6db15dcd7d80d7d5bc406c4970a3478ada8899b329198eb61c193fb62
 75aa8ca340344a75a862aeb92eee1ce032fd950b47d7704a3876923b4ad6284

16 PTX 4bf4a09c4dbe8b4397184b7471360c9564880aedddb9baa4af2e75394b08cd32

17 PTX ff479c57a07d3eab5d54de5f9738b8d27f27a9f0ab11799d7b7ffefb2704c95c

18 PTX 6ad12c39f1e867a4b7b1d7818a4b753dfd2a89ccb45e001a03a867b187f225dd

19 CTX fa762a3680b76007928ed4a4f49a9456031b704782e65e16cecb54ed7d017b5e9

20 CTX 18abd67b338e81078f21edb7868d901ebe9c731a7c18b5e6dec1d6a72e078ac9

21 CTX a4262f860beefa14f4e821018272e411a951502b6e79066e84252c3346f3aa62

22 CTX 344351a291d4bedc7a07618bdea2af63145cc7a4b8d4070691ae890cd65733e7

23 CTX 946e9021a1dffc4c59f159425ee6d50ca9b135fa6162cea18a939838dc000fb3

24 CTX 86fad086acce5ac07cb2ece7fd580b00cfa5e98589631dc25e8e2a3daf2ffdec

25 CTX 26531659912c9d8f7a15e5865ea8fb5816d6207052bd7128cd743c12c8118791

26 CTX a4736811935eb982a532349e31dd401e0b660a568cb1a4711f552f55ded59f1f

27 CTX 15bf7196b3ca12a91e488ef59d64f3a02bf45239499ac6176ae321c4a211ec54

28 CTX 5365971c5d3f4f09d4eb139bdfd2073d33180b21002b65cc9865e76cb24cd92c

29 CTX 874c24c18350399a936ab3637079295d76c41776b94efce3a0ef7206b151105

30 CTX 19655c956cbd8b2489405ee2b09a6b6eebe0c53790a12a8998378b33a5b71159

31 CTX 625f4ba49d2a2fdb59fbf0897bc7aabd8d707dc140a80f0f309f835d3da54ab

32 CTX 584e501dfa0ee97fec543f74186a802b9a37adb3e8291eca04d66520d229e60

33 CTX 401e7282bef486ae059aa70696e0e305d777140a7a883ecdcb69b9ff938e8a42

34 CTX 31864c69ca2c2043bed007ff3e605e014bcf518138dc3a25c5e236171a2d01d6

35

36 Vector 7

37 Key1 27182818284590452353602874713526

38 Key2 31415926535897932384626433832795

39 Data Unit Sequence Number fd

40

41 PTX 8e41b78c390b5af9d758bb214a67e9f6bf7727b09ac6124084c37611398fa45d

42 PTX aad94868600ed391fb1acd4857a95b466e62ef9f4b377244d1c152e7b30d731a

43 PTX ad30c716d214b707aed99eb5b5e580b3e887cf7497465651d4b60e6042051da3

44 PTX 693c3b78c14489543be8b6ad0ba629565bba202313ba7b0d0c94a3252b676f46

45 PTX cc02ce0f8a7d34c0ed229129673c1f61aed579d08a9203a25aac3a77e9db6026

46 PTX 7996db38df637356d9dcd1632e369939f2a29d89345c66e05066f1a3677aef18

47 PTX dea4113faeb629e46721a66d0a7e785d3e29af2594eb67dfa982affe0aac058f

48 PTX 6e15864269b135418261fc3afb089472cf68c45dd7f231c6249ba0255e1e0338

49 PTX 33fc4d00a3fe02132d7bc3873614b8aee34273581ea0325c81f0270affa13641

50 PTX d052d36f0757d484014354d02d6883ca15c24d8c3956b1bd027bcf41f151fd80

51 PTX 23c5340e5606f37e90fdb87c86fb4fa634b3718a30bace06a66eaf8f63c4aa3b

52 PTX 637826a87fe8cfa44282e92cb1615af3a28e53bc74c7cba1a0977be9065d0c1a

53 PTX 5dec6c54ae38d37f37aa35283e048e5530a85c4e7a29d7b92ec0c3169cdf2a80

54 PTX 5c7604bce60049b9fb7b8eaac10f51ae23794ceba68bb58112e293b9b692ca72

55 PTX 1b37c662f8574ed4dba6f88e170881c82cddc1034a0ca7e284bf0962b6b26292

56 PTX d836fa9f73c1ac770eeff0f2d3a1eaf61d3e03555fd424eedd67e18a18094f888

57 CTX d55f684f81f4426e9fd9e2a5ff02df2ac896af63962888a97910c1379e20b0a3

58 CTX b1db613fb7fe2e07004329ea5c22bfd33e3dbe4cf58cc608c2c26c19a2e2fe22

59 CTX f98732c2b5cb844cc6c0702d91e1d50fc4382a7eba5635cd602432a2306ac4ce

60 CTX 82f8d70c8d9bc15f918fe71e74c622d5cf71178bf6e0b9cc9f2b41dd8dbe441c

61 CTX 41cd0c73a6dc47a348f6702f9d0e9b1b1431e948e299b9ec2272ab2c5f0c7be8

62 CTX 6affa5dec87a0bee81d3d50007edaa2bcfccb35605155ff36ed8edd4a40dcd4b

63 CTX 243acd11b2b987bdfbaf91a7cac27e9c5aea525ee53de7b2d3332c8644402b82

64 CTX 3e94a7db26276d2d23aa07180f76b4fd29b9c0823099c9d62c519880aee7e969

```

1 CTX 7617c1497d47bf3e571950311421b6b734d38b0db91eb85331b91ea9f61530f5
2 CTX 4512a5a52a4bad589eb69781d537f23297bb459bdad2948a29e1550bf4787e0b
3 CTX e95bb173cf5fab17dab7a13a052a63453d97ccec1a321954886b7a1299faaeec
4 CTX ae35c6eaaca753b041b5e5f093bf83397fd21dd6b3012066fcc058cc32c3b09d
  7562dee29509b5839392c9ff05f51f3166aaac4ac5f238038a3045e6f72e48ef
  0fe8bc675e82c318a268e43970271bf119b81bf6a982746554f84e72b9f00280
  a320a08142923c23c883423ff949827f29bbacdc1ccdb04938ce6098c95ba6b3
  2528f4ef78eed778b2e122ddf1cbdd11d1c0a6783e011fc536d63d053260637
  or 8
  27182818284590452353602874713526
  31415926535897932384626433832795
  Unit Sequence Number fe
  15 PTX d55f684f81f4426e9fde92a5ff02df2ac896af63962888a97910c1379e20b0a3
16 PTX b1db613fb7fe2e07004329ea5c22bfd33e3dbe4cf58cc608c2c26c19a2e2fe22
17 PTX f98732c2b5cb844cc6c0702d91e1d50fc4382a7eba5635cd602432a2306ac4ce
18 PTX 82f8d70c8d9bc15f918fe71e74c622d5cf71178bf6e0b9cc9f2b41dd8dbe441c
19 PTX 41cd0c73a6dc47a348f6702f9d0e9b1b1431e948e299b9ec2272ab2c5f0c7be8
20 PTX 6affa5dec87a0bee81d3d50007edaa2bcfccb35605155ff36ed8edd4a40dcd4b
21 PTX 243acd11b2b987bdbfaf91a7cac27e9c5aea525ee53de7b2d3332c8644402b82
22 PTX 3e94a7db26276d2d23aa07180f76b4fd29b9c0823099c9d62c519880aee7e969
23 PTX 7617c1497d47bf3e571950311421b6b734d38b0db91eb85331b91ea9f61530f5
24 PTX 4512a5a52a4bad589eb69781d537f23297bb459bdad2948a29e1550bf4787e0b
25 PTX e95bb173cf5fab17dab7a13a052a63453d97ccec1a321954886b7a1299faaeec
26 PTX ae35c6eaaca753b041b5e5f093bf83397fd21dd6b3012066fcc058cc32c3b09d
27 PTX 7562dee29509b5839392c9ff05f51f3166aaac4ac5f238038a3045e6f72e48ef
28 PTX 0fe8bc675e82c318a268e43970271bf119b81bf6a982746554f84e72b9f00280
29 PTX a320a08142923c23c883423ff949827f29bbacdc1ccdb04938ce6098c95ba6b3
30 PTX 2528f4ef78eed778b2e122ddf1cbdd11d1c0a6783e011fc536d63d053260637
31 CTX 72efc1ebfe1ee25975a6eb3aa8589dda2b261f1c85bdab442a9e5b2dd1d7c395
32 CTX 7a16fc08e526d4b1223f1b1232a11af274c3d70dac57f83e0983c498f1a6f1ae
33 CTX cb021c3e70085a1e527f1ce41ee5911a82020161529cd82773762daf5459de94
34 CTX a0a82adae7e1703c808543c29ed6fb32d9e004327c1355180c995a07741493a0
35 CTX 9c21ba01a387882da4f62534b87bb15d60d197201c0fd3bf30c1500a3ecfecdd
36 CTX 66d8721f90bcc4c17ee925c61b0a03727a9c0d5f5ca462fbfa0af1c2513a9d9d
37 CTX 4b5345bd27a5f6e653f751693e6b6a2b8ead57d511e00e58c45b7b8d005af792
38 CTX 88f5c7c22fd4f1bf7a898b03a5634c6a1ae3f9fae5de4f296a2896b23e7ed43e
39 CTX d14fa5a2803f4d28f0d3ffcf24757677aebdb47bb388378708948a8d4126ed18
40 CTX 39e0da29a537a8c198b3c66ab00712dd261674bf45a73d67f76914f830ca014b
41 CTX 65596f27e4cf62de66125a5566df9975155628b400fbfb3a29040ed50faffdbb
42 CTX 18aace7c5c44693260aab386c0a37b11b114f1c415aebb653be468179428d43a
43 CTX 4d8bc3ec38813eca30a13cf1bb18d524f1992d44d8b1a42ea30b22e6c95b199d
44 CTX 8d182f884b09d059585c31ad691fa0619ff038aca2c39a943421157361717c4
45 CTX 9d322028a74648113bd8c9d7ec77cf3c89c1ec8718ceff8516d96b34c3c614f1
46 CTX 0699c9abc4ed0411506223bea16af35c883accdbe1104eef0cfdb54e12fb230a
47
48 Vector 9
49 Key1 27182818284590452353602874713526
50 Key2 31415926535897932384626433832795
51 Data Unit Sequence Number ff
52
53 PTX 72efc1ebfe1ee25975a6eb3aa8589dda2b261f1c85bdab442a9e5b2dd1d7c395
54 PTX 7a16fc08e526d4b1223f1b1232a11af274c3d70dac57f83e0983c498f1a6f1ae
55 PTX cb021c3e70085a1e527f1ce41ee5911a82020161529cd82773762daf5459de94
56 PTX a0a82adae7e1703c808543c29ed6fb32d9e004327c1355180c995a07741493a0
57 PTX 9c21ba01a387882da4f62534b87bb15d60d197201c0fd3bf30c1500a3ecfecdd
58 PTX 66d8721f90bcc4c17ee925c61b0a03727a9c0d5f5ca462fbfa0af1c2513a9d9d
59 PTX 4b5345bd27a5f6e653f751693e6b6a2b8ead57d511e00e58c45b7b8d005af792
60 PTX 88f5c7c22fd4f1bf7a898b03a5634c6a1ae3f9fae5de4f296a2896b23e7ed43e
61 PTX d14fa5a2803f4d28f0d3ffcf24757677aebdb47bb388378708948a8d4126ed18
62 PTX 39e0da29a537a8c198b3c66ab00712dd261674bf45a73d67f76914f830ca014b
63 PTX 65596f27e4cf62de66125a5566df9975155628b400fbfb3a29040ed50faffdbb
64 PTX 18aace7c5c44693260aab386c0a37b11b114f1c415aebb653be468179428d43a

```

1 PTX 4d8bc3ec38813eca30a13cf1bb18d524f1992d44d8b1a42ea30b22e6c95b199d
 2 PTX 8d182f8840b09d059585c31ad691fa0619ff038aca2c39a943421157361717c4
 3 PTX 9d322028a74648113bd8c9d7ec77cf3c89c1ec8718ceff8516d96b34c3c614f1
 4 PTX 0699c9abc4ed0411506223bea16af35c883accdbe1104eef0cfdb54e12fb230a
 3260ae8dad1f4a32c5cafe3ab0eb95549d461a67ceb9e5aa2d3afb62dece0553
 193ba50c75be251e08d1d08f1088576c7efdfaaf3f459559571e12511753b07a
 f073f35da06af0ce0bbf6b8f5ccc5cea500ec1b211bd51f63b606bf6528796ca
 12173ba39b8935ee44ccce646f90a45bf9ccc567f0ace13dc2d53beedc81f58
 02e41179dddf0d5a5c42f5d8506c1a5d2f8f59f3ea873cbcd0eec19acbf32542
 3bd3dcb8c2b1bf1d1eae0eba7f0698e4314fbeb2f1566d1b9253008cbccf45a
 2b0d9c5c9c21474f4076e02be26050b99dee4fd68a4cf890e496e4fcae7b70f9
 4ea5a9062da0daeba1993d2ccd1dd3c244b8428801495a58b216547e7e847c46
 d1d756377b6242d2e5fb83bf752b54e0df71e889f3a2bb0f4c10805bf3c59037
 6e3c24e22ff5f77fa965577375325cea5d920db94b9c336b455f6e894c01866f
 e9fbb8c8d3f70a2957285f6dfb5dcd8cbf54782f8fe7766d4723819913ac7734
 16 CTX 21e3a31095866bad22c86a6036b2518b2059b4229d18c8c2ccbd906c6cc6e82
 17 CTX 464ee57b0db0beebcb1dc645325bf3e665ef7251082c88ebb1cf203bd779fdd3
 18 CTX 8675713c8daadd17e1cabee432b09787b6dddf3304e38b731b45df5df51b78fcf
 19 CTX b3d32466028d0ba36555e7e11ab0ee0666061d1645d962444bc47a38188930a8
 20 CTX 4b4d561395c73c087021927ca638b7afc8a8679ccb84c26555440ec7f10445cd
 21
 22

XTS-AES-256 applied for a data unit of 512 bytes

Vector 10

Key1 2718281828459045235360287471352662497757247093699959574966967627

Key2 3141592653589793238462643383279502884197169399375105820974944592

Data Unit Sequence Number ff

30 PTX 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
 31 PTX 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
 32 PTX 404142434445464748494a4b4c4d4e4f505152535455565758595a5b5c5d5e5f
 33 PTX 606162636465666768696a6b6c6d6e6f707172737475767778797a7b7c7d7e7f
 34 PTX 808182838485868788898a8b8c8d8e8f909192939495969798999a9b9c9d9e9f
 35 PTX a0a1a2a3a4a5a6a7a8a9aaabacadaeafb0b1b2b3b4b5b6b7b8b9babbbcbdbefb
 36 PTX c0c1c2c3c4c5c6c7c8c9cacbcccdcecf0d1d2d3d4d5d6d7d8d9dadbdcdddedf
 37 PTX e0e1e2e3e4e5e6e7e8e9eaebeceedeef0f1f2f3f4f5f6f7f8f9fafbfcfdfeff
 38 PTX 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
 39 PTX 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
 40 PTX 404142434445464748494a4b4c4d4e4f505152535455565758595a5b5c5d5e5f
 41 PTX 606162636465666768696a6b6c6d6e6f707172737475767778797a7b7c7d7e7f
 42 PTX 808182838485868788898a8b8c8d8e8f909192939495969798999a9b9c9d9e9f
 43 PTX a0a1a2a3a4a5a6a7a8a9aaabacadaeafb0b1b2b3b4b5b6b7b8b9babbbcbdbefb
 44 PTX c0c1c2c3c4c5c6c7c8c9cacbcccdcecf0d1d2d3d4d5d6d7d8d9dadbdcdddedf
 45 PTX e0e1e2e3e4e5e6e7e8e9eaebeceedeef0f1f2f3f4f5f6f7f8f9fafbfcfdfeff
 46 CTX 1c3b3a102f770386e4836c99e370cf9bea00803f5e482357a4ae12d414a3e63b
 47 CTX 5d31e276f8fe4a8d66b317f9ac683f44680a86ac35adfc3345befecb4bb188fd
 48 CTX 5776926c49a3095eb108fd1098baec70aaa66999a72a82f27d848b21d4a741b0
 49 CTX c5cd4d5fff9dac89aeba122961d03a757123e9870f8acf1000020887891429ca
 50 CTX 2a3e7a7d7df7b10355165c8b9a6d0a7de8b062c4500dc4cd120c0f7418dae3d0
 51 CTX b5781c34803fa75421c790dfe1de1834f280d7667b327f6c8cd7557e12ac3a0f
 52 CTX 93ec05c52e0493ef31a12d3d9260f79a289d6a379bc70c50841473d1a8cc81ec
 53 CTX 583e9645e07b8d9670655ba5bbcfec6dc3966380ad8fecb17b6ba02469a020a
 54 CTX 84e18e8f84252070c13e9f1f289be54fbc481457778f616015e1327a02b140f1
 55 CTX 505eb309326d68378f8374595c849d84f4c333ec4423885143cb47bd71c5edae
 56 CTX 9be69a2ffecbe1bec9de244fbe15992b11b77c040f12bd8f6a975a44a0f90c29
 57 CTX a9abc3d4d893927284c58754cce294529f8614dcd2aba991925fedc4ae74ffac
 58 CTX 6e333b93eb4aff0479da9a410e4450e0dd7ae4c6e2910900575da401fc07059f
 59 CTX 645e8b7e9bfdef33943054ff84011493c27b3429eadeb4ed5376441a77ed4385
 60 CTX 1ad77f16f541df269d50d6a5f14fb0aab1cbb4c1550be97f7ab4066193c4caa
 61 CTX 773dad38014bd2092fa755c824bb5e54c4f36ffda9fcea70b9c6e693e148c151
 62 GEN
 63

```

1  Vector 11
2  Key1 2718281828459045235360287471352662497757247093699959574966967627
3  Key2 3141592653589793238462643383279502884197169399375105820974944592
4  Data Unit Sequence Number ffff

15 PTX 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
16 PTX 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
17 PTX 404142434445464748494a4b4c4d4e4f505152535455565758595a5b5c5d5e5f
18 PTX 606162636465666768696a6b6c6d6e6f707172737475767778797a7b7c7d7e7f
19 PTX 808182838485868788898a8b8c8d8e8f909192939495969798999a9b9c9d9e9f
20 PTX a0a1a2a3a4a5a6a7a8a9aaabacadaeafb0b1b2b3b4b5b6b7b8b9babbbcbdbef
21 PTX c0c1c2c3c4c5c6c7c8c9cacbcccdcecf0d0d1d2d3d4d5d6d7d8d9dadbdcddeff
22 CTX e0e1e2e3e4e5e6e7e8e9eaebeceedeef0f1f2f3f4f5f6f7f8f9fafbfcfdfeff
23 CTX 77a31251618a15e6b92d1d66df7b50b50bad552305ba0217a610688eff7e11
24 CTX e1d0225438e093242d6db274fde801d4cae06f2092c728b2478559df58e837c2
25 CTX 469ee4a4fa794e4bbc7f39bc026e3cb72c33b0888f25b4acf56a2a9804f1ce6d
26 CTX 3d6e1dc6ca181d4b546179d5554aa7760c40d06741539c7e3cd9d2f6650b201
27 CTX 3fd0eeb8c2b8e3d8d240ccae2d4c98320a7442e1c8d75a42d6e6cfa4c2eca179
28 CTX 8d158c7aecdf82490f24bb9b38e108bcd12c3faf9a21141c3613b58367f922a
29 CTX aa26cd22f23d708dae699ad7cb40a8ad0b6e2784973dcb605684c08b8d6998c6
30 CTX 9aac049921871ebbb65301a4619ca80ecb485a31d744223ce8ddc2394828d6a80
31 CTX 470c092f5ba413c3378fa6054255c6f9df4495862bbb3287681f931b687c888a
32 CTX bf844dfc8fc28331e579928cd12bd2390ae123cf03818d14dedde5c0c24c8ab0
33 CTX 18bfca75ca096f2d531f3d1619e785f1ada437cab92e980558b3dce1474afb75
34 CTX bfeadbfbf8f54cb2618e0244c9ac0d3c66fb51598cd2db11f9be39791abe447c63
35 CTX 094f7c453b7ff87cb5bb36b7c79efb0872d17058b83b15ab0866ad8a58656c5a
36 CTX 7e20dbdf308b2461d97c0ec0024a2715055249cf3b478ddd4740de654f75ca68
37 CTX 6e0d7345c69ed50cdc2a8b332b1f8824108ac937eb050585608ee734097fc090
38 CTX 54fbff89eeaeaa791f4a7ab1f9868294a4f9e27b42af8100cb9d59cef9645803
39  Vector 12
40 Key1 2718281828459045235360287471352662497757247093699959574966967627
41 Key2 3141592653589793238462643383279502884197169399375105820974944592
42 Data Unit Sequence Number ffffff
43
44 PTX 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
45 PTX 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
46 PTX 404142434445464748494a4b4c4d4e4f505152535455565758595a5b5c5d5e5f
47 PTX 606162636465666768696a6b6c6d6e6f707172737475767778797a7b7c7d7e7f
48 PTX 808182838485868788898a8b8c8d8e8f909192939495969798999a9b9c9d9e9f
49 PTX a0a1a2a3a4a5a6a7a8a9aaabacadaeafb0b1b2b3b4b5b6b7b8b9babbbcbdbef
50 PTX c0c1c2c3c4c5c6c7c8c9cacbcccdcecf0d0d1d2d3d4d5d6d7d8d9dadbdcddeff
51 PTX e0e1e2e3e4e5e6e7e8e9eaebeceedeef0f1f2f3f4f5f6f7f8f9fafbfcfdfeff
52 PTX 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
53 PTX 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
54 PTX 404142434445464748494a4b4c4d4e4f505152535455565758595a5b5c5d5e5f
55 PTX 606162636465666768696a6b6c6d6e6f707172737475767778797a7b7c7d7e7f
56 PTX 808182838485868788898a8b8c8d8e8f909192939495969798999a9b9c9d9e9f
57 PTX a0a1a2a3a4a5a6a7a8a9aaabacadaeafb0b1b2b3b4b5b6b7b8b9babbbcbdbef
58 PTX c0c1c2c3c4c5c6c7c8c9cacbcccdcecf0d0d1d2d3d4d5d6d7d8d9dadbdcddeff
59 PTX e0e1e2e3e4e5e6e7e8e9eaebeceedeef0f1f2f3f4f5f6f7f8f9fafbfcfdfeff
60 CTX e387aaa58ba483afa7e8eb469778317ecf4cf573aa9d4eac23f2cdf914e4e200
61 CTX a8b490e42ee646802dc6ee2b471b278195d60918ecec44bf79966f83faba049
62 CTX 9298ebc699c08634715a320bb4f075d622e74c8c932004f25b41e361025b5a8
63 CTX 7815391f6108fc4afa6a05d930c6ba68a128a55705d415985832fdeaae6c8e1
64 CTX 9110e84d1b1f199a2692119edc96132658f09da7c623efcec712537a3d94c0bf

```

```

1 CTX 5d7e352ec94ae5797fdb377dc1551150721adf15bd26a8efc2fcaad56881fa9e
2 CTX 62462c28f30ae1ceaca93c345cf243b73f542e2074a705bd2643bb9f7cc79bb6
3 CTX e7091ea6e232df0f9ad0d6cf502327876d82207abf2115cdac6d5a48f6c1879
4 CTX a65b115f0f8b3cb3c59d15dd8c769bc014795a1837f3901b5845eb491adfefe0
  97b1fa30a12fc1f65ba22905031539971a10f2f36c321bb51331cdefb39e3964
  c7ef079994f5b69b2edd83a71ef549971ee93f44eac3938fcd61d01fa71799d
  a3a8091c4c48aa9ed263ff0749df95d44fef6a0bb578ec69456aa5408ae32c7a
  f08ad7ba8921287e3bbe31b767be06a0e705c864a769137df28292283ea81a2
  480241b44d9921cdbc1bc28dc1fda114bd8e5217ac9d8ebafa720e9da4f9ace
  231cc949e5b96fe76ffc21063fddc83a6b8679c00d35e09576a875305bed5f36
  ed242c8900dd1fa965bc950dfce09b132263a1eef52dd6888c309f5a7d712826
  or 13
  2718281828459045235360287471352662497757247093699959574966967627
  3141592653589793238462643383279502884197169399375105820974944592
  Data Unit Sequence Number ffffffff
16
17
18 PTX 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
19 PTX 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
20 PTX 404142434445464748494a4b4c4d4e4f505152535455565758595a5b5c5d5e5f
21 PTX 606162636465666768696a6b6c6d6e6f707172737475767778797a7b7c7d7e7f
22 PTX 808182838485868788898a8b8c8d8e8f909192939495969798999a9b9c9d9e9f
23 PTX a0a1a2a3a4a5a6a7a8a9aaabacadaeafb0b1b2b3b4b5b6b7b8b9babbbcbdbbfb
24 PTX c0c1c2c3c4c5c6c7c8c9cacbcccdcecfdd0d1d2d3d4d5d6d7d8d9dadbdcddefff
25 PTX e0e1e2e3e4e5e6e7e8e9eaebeceedeef0f1f2f3f4f5f6f7f8f9fafbfcfdfefff
26 PTX 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
27 PTX 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
28 PTX 404142434445464748494a4b4c4d4e4f505152535455565758595a5b5c5d5e5f
29 PTX 606162636465666768696a6b6c6d6e6f707172737475767778797a7b7c7d7e7f
30 PTX 808182838485868788898a8b8c8d8e8f909192939495969798999a9b9c9d9e9f
31 PTX a0a1a2a3a4a5a6a7a8a9aaabacadaeafb0b1b2b3b4b5b6b7b8b9babbbcbdbbfb
32 PTX c0c1c2c3c4c5c6c7c8c9cacbcccdcecfdd0d1d2d3d4d5d6d7d8d9dadbdcddefff
33 PTX e0e1e2e3e4e5e6e7e8e9eaebeceedeef0f1f2f3f4f5f6f7f8f9fafbfcfdfefff
34 CTX bf53d2dade78e822a4d949a9bc6766b01b06a8ef70d26748c6a7fc36d80ae4c5
35 CTX 520f7c4ab0ac8544424fa405162fef5a6b7f229498063618d39f0003cb5fb8d1
36 CTX c86b643497da1ff945c8d3bedeca4f479702a7a735f043ddb1d6aaade3c4a0ac
37 CTX 7ca7f3fa5279bef56f82cd7a2f38672e824814e10700300a055e1630b8f1cb0e
38 CTX 919f5e942010a416e2bf48cb46993d3cb6a51c19bacf864785a00bc2ecff15d3
39 CTX 50875b246ed53e68be6f55bd7e05cfc2b2ed6432198a6444b6d8c247fab941f5
40 CTX 69768b5c429366f1d3f00f0345b96123d56204c01c63b22ce78baf116e525ed9
41 CTX 0fdea39fa469494d3866c31e05f295ff21fea8d4e6e13d67e47ce722e9698a1c
42 CTX 1048d68ebcde76b86fcf976eab8aa9790268b7068e017a8b9b749409514f1053
43 CTX 027fd16c3786ea1bac5f15cb79711ee2abe82f5cf8b13ae73030ef5b9e4457e7
44 CTX 5d1304f988d62dd6fc4b94ed38ba831da4b7634971b6cd8ec325d9c61c00f1df
45 CTX 73627ed3745a5e8489f3a95c69639c32cd6e1d537a85f75cc844726e8a72fc00
46 CTX 77ad22000f1d5078f6b866318c668f1ad03d5a5fced5219f2eabbd0aa5c0f460
47 CTX d183f04404a0d6f469558e81fab24a167905ab4c7878502ad3e38fdbe62a4155
48 CTX 6cec37325759533ce8f25f367c87bb5578d667ae93f9e2fd99bcbcf5f2fbba88c
49 CTX f6516139420fcff3b7361d86322c4bd84c82f335abb152c4a93411373aaa8220
50
51 Vector 14
52 Key1 2718281828459045235360287471352662497757247093699959574966967627
53 Key2 3141592653589793238462643383279502884197169399375105820974944592
54 Data Unit Sequence Number ffffffff
55
56 PTX 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
57 PTX 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
58 PTX 404142434445464748494a4b4c4d4e4f505152535455565758595a5b5c5d5e5f
59 PTX 606162636465666768696a6b6c6d6e6f707172737475767778797a7b7c7d7e7f
60 PTX 808182838485868788898a8b8c8d8e8f909192939495969798999a9b9c9d9e9f
61 PTX a0a1a2a3a4a5a6a7a8a9aaabacadaeafb0b1b2b3b4b5b6b7b8b9babbbcbdbbfb
62 PTX c0c1c2c3c4c5c6c7c8c9cacbcccdcecfdd0d1d2d3d4d5d6d7d8d9dadbdcddefff
63 PTX e0e1e2e3e4e5e6e7e8e9eaebeceedeef0f1f2f3f4f5f6f7f8f9fafbfcfdfefff
64 PTX 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f

```

```

1 PTX 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
2 PTX 404142434445464748494a4b4c4d4e4f505152535455565758595a5b5c5d5e5f
3 PTX 606162636465666768696a6b6c6d6e6f707172737475767778797a7b7c7d7e7f
4 PTX 808182838485868788898a8b8c8d8e8f909192939495969798999a9b9c9d9e9f
   a0a1a2a3a4a5a6a7a8a9aaabacadaeafb0b1b2b3b4b5b6b7b8b9babbbcbdbbfbf
   c0c1c2c3c4c5c6c7c8c9cacbcccdcecfdd0d1d2d3d4d5d6d7d8d9dadbdcdddedf
   e0e1e2e3e4e5e6e7e8e9eaebeceedeef0f1f2f3f4f5f6f7f8f9fafbfcfdfeff
   64497e5a831e4a932c09be3e5393376daa599548b816031d224bbf50a818ed23
   50eae7e96087c8a0db51ad290bd00c1ac1620857635bf246c176ab463be30b80
   3da548081ac847b158e1264be25bb0910bbc92647108089415d45fab1b3d2604
   e8a8eff1ae4020cfa39936b66827b23f371b92200be90251e6d73c5f86de5fd4
   a950781933d79a28272b782a2ec313efdfcc0628f43d744c2dc2ff3dcb66999b
   50c7ca895b0c64791eeaa5f29499fb1c026f84ce5b5c72ba1083cddb5ce45434
   631665c333b60b11593fb253c5179a2c8db813782a004856a1653011e93fb6d8
   76c18366dd8683f53412c0c180f9c848592d593f8609ca736317d356e13e2bff
15 CTX 3a9f59cd9aeb19cd482593d8c46128bb32423b37a9adfb482b99453fbe25a41b
16 CTX f6feb4aa0bef5ed24bf73c762978025482c13115e4015aac992e5613a3b5c2f6
17 CTX 85b84795cb6e9b2656d8c88157e52c42f978d8634c43d06fea928f2822e465aa
18 CTX 6576e9bf419384506cc3ce3c54ac1a6f67dc66f3b30191e698380bc999b05abc
19 CTX e19dc0c6dcc2dd001ec535ba18deb2df1a101023108318c75dc98611a09dc48a
20 CTX 0acdec676fabdf222f07e026f059b672b56e5cbc8e1d21bbd867dd9272120546
21 CTX 81d70ea737134cdfce93b6f82ae22423274e58a0821cc5502e2d0ab4585e94de
22 CTX 6975be5e0b4efce51cd3e70c25a1fbbbd609d273ad5b0d59631c531f6a0a57b9
23
24
25 XTS-AES-128 applied for a data unit that is not a multiple of 16 bytes
26
27 Vector 15
28 Key1 fffefdfcfbfaf9f8f7f6f5f4f3f2f1f0
29 Key2 bfbdbcbcbab9b8b7b6b5b4b3b2b1b0
30 Data unit sequence number 9a78563412
31
32 PTX 000102030405060708090a0b0c0d0e0f10
33 CTX 6c1625db4671522d3d7599601de7ca09ed
34
35 Vector 16
36 Key1 fffefdfcfbfaf9f8f7f6f5f4f3f2f1f0
37 Key2 bfbdbcbcbab9b8b7b6b5b4b3b2b1b0
38 Data unit sequence number 9a78563412
39
40 PTX 000102030405060708090a0b0c0d0e0f1011
41 CTX d069444b7a7e0cab09e24447d24deb1fedbf
42
43 Vector 17
44 Key1 fffefdfcfbfaf9f8f7f6f5f4f3f2f1f0
45 Key2 bfbdbcbcbab9b8b7b6b5b4b3b2b1b0
46 Data unit sequence number 9a78563412
47
48 PTX 000102030405060708090a0b0c0d0e0f101112
49 CTX e5df1351c0544ba1350b3363cd8ef4beedbf9d
50
51 Vector 18
52 Key1 fffefdfcfbfaf9f8f7f6f5f4f3f2f1f0
53 Key2 bfbdbcbcbab9b8b7b6b5b4b3b2b1b0
54 Data unit sequence number 9a78563412
55
56 PTX 000102030405060708090a0b0c0d0e0f10111213
57 CTX 9d84c813f719aa2c7be3f66171c7c5c2edbf9dac
58
59 Vector 19
60 Key1 e0e1e2e3e4e5e6e7e8e9eaebeceedeef
61 Key2 c0c1c2c3c4c5c6c7c8c9cacbcccdcecf
62 Data unit sequence number 21436587a9
63
64 PTX 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f

```

t(평문)
 ext(암호문)
 mask
 XTS key
 uence
 a unit 순번
 r 값은
 16로 유지

1	PTX	202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
2	PTX	404142434445464748494a4b4c4d4e4f505152535455565758595a5b5c5d5e5f
3	PTX	606162636465666768696a6b6c6d6e6f707172737475767778797a7b7c7d7e7f
4	PTX	808182838485868788898a8b8c8d8e8f909192939495969798999a9b9c9d9e9f
		a0a1a2a3a4a5a6a7a8a9aaabacadaeafb0b1b2b3b4b5b6b7b8b9babbbcbdbbfb
		c0c1c2c3c4c5c6c7c8c9cacbcccdcecfdd0d1d2d3d4d5d6d7d8d9dadbdcddeedf
tt(평문)		e0e1e2e3e4e5e6e7e8e9eaebeceedeefeff0f1f2f3f4f5f6f7f8f9fafbfcfdfeff
ext(암호문)		000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
mask		202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
XTS key		404142434445464748494a4b4c4d4e4f505152535455565758595a5b5c5d5e5f
uence		606162636465666768696a6b6c6d6e6f707172737475767778797a7b7c7d7e7f
a unit 순번		808182838485868788898a8b8c8d8e8f909192939495969798999a9b9c9d9e9f
r 값은		a0a1a2a3a4a5a6a7a8a9aaabacadaeafb0b1b2b3b4b5b6b7b8b9babbbcbdbbfb
문 유지		c0c1c2c3c4c5c6c7c8c9cacbcccdcecfdd0d1d2d3d4d5d6d7d8d9dadbdcddeedf
		e0e1e2e3e4e5e6e7e8e9eaebeceedeefeff0f1f2f3f4f5f6f7f8f9fafbfcfdfeff
15	PTX	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
16	CTX	38b45812ef43a05bd957e545907e223b954ab4aaf088303ad910eadf14b42be6
17	CTX	8b2461149d8c8ba85f992be970bc621f1b06573f63e867bf5875acafa04e42cc
18	CTX	bd7bd3c2a0fb1fff791ec5ec36c66ae4ac1e806d81fbf709dbe29e471fad3854
19	CTX	9c8e66f5345d7c1eb94f405d1ec785cc6f6a68f6254dd8339f9d84057e01a177
20	CTX	41990482999516b5611a38f41bb6478e6f173f320805dd71b1932fc333cb9ee3
21	CTX	9936beea9ad96fa10fb4112b901734ddad40bc1878995f8e11aee7d141a2f5d4
22	CTX	8b7a4e1e7f0b2c04830e69a4fd1378411c2f287edf48c6c4e5c247a19680f7fe
23	CTX	41cefbd49b582106e3616cbb4dfb2344b2ae9519391f3e0fb4922254b1d6d2d
24	CTX	19c6d4d537b3a26f3bcc51588b32f3eca0829b6a5ac72578fb814fb43cf80d64
25	CTX	a233e3f997a3f02683342f2b33d25b492536b93becb2f5e1a8b82f5b88334272
26	CTX	9e8ae09d16938841a21a97fb543eea3bbbf59f13c1a18449e398701c1ad51648
27	CTX	346cbc04c27bb2da3b93a1372ccae548fb53bee476f9e9c91773b1bb19828394
28	CTX	d55d3e1a20ed69113a860b6829ffa847224604435070221b257e8dff783615d2
29	CTX	cae4803a93aa4334ab482a0afac9c0aeda70b45a481df5dec5df8cc0f423c77a
30	CTX	5fd46cd312021d4b438862419a791be03bb4d97c0e59578542531ba466a83baf
31	CTX	92cefc151b5cc1611a167893819b63fb8a6b18e86de60290fa72b797b0ce59f3

Annex C (informative)

Pseudocode for XTS-AES-128 and XTS-AES-256 Encryption

C.1 16 byte의 배수 크기 data unit 암호화

학습 주석 | 원문의 pseudocode는 구현 비교를 위해 function/variable name을 그대로 유지합니다. English comment는 아래에서 한국어 뜻을 함께 설명합니다.

```
#define GF_128_FDBK 0x87
#define AES_BLK_BYTES 16
void XTS_EncryptSector(
    AES_Key &k2,      // key used for tweaking
    AES_Key &k1,      // key used for "ECB" encryption
    u64b S,          // data unit number (64 bits)
    uint N,          // sector size, in bytes
    const u08b *pt,  // plaintext sector input data
    u08b *ct         // ciphertext sector output data
) {
    uint i,j;
    u08b T[AES_BLK_BYTES];
    u08b x[AES_BLK_BYTES];
    u08b Cin,Cout;

    assert(N % AES_BLK_BYTES == 0);
    for (j=0;j<AES_BLK_BYTES;j++) {
        T[j] = (u08b)(S & 0xFF);
        S = S >> 8;
    }
    AES_ECB_Encrypt(k2,T);

    for (i=0;i<N;i+=AES_BLK_BYTES) {
        for (j=0;j<AES_BLK_BYTES;j++) x[j] = pt[i+j] ^ T[j];
        AES_ECB_Encrypt(k1,x);
        for (j=0;j<AES_BLK_BYTES;j++) ct[i+j] = x[j] ^ T[j];

        Cin = 0;
        for (j=0;j<AES_BLK_BYTES;j++) {
            Cout = (T[j] >> 7) & 1;
            T[j] = ((T[j] << 1) + Cin) & 0xFF;
            Cin = Cout;
        }
        if (Cout) T[0] ^= GF_128_FDBK;
    }
}
```

주석 해석: k2는 tweak 생성용 key, k1은 실제 data AES encryption용 key, S는 data unit/sector number, N은 byte 단위 sector size, pt/ct는 plaintext/ciphertext buffer이다. T는 현재 block tweak, x는 XOR/AES 작업용 buffer이다. 마지막 loop는 $T \leftarrow \alpha T$ 를 LFSR 형태로 구현하며 carry가 끝까지 나가면 T[0]에 0x87을 XOR한다.

C.2 16 byte의 배수가 아닌 data unit 암호화

마지막 partial block을 처리하기 위해 ciphertext stealing을 추가한다. 아래 pseudocode는 원문 identifier를 유지한다.

```
#define GF_128_FDBK 0x87
#define AES_BLK_BYTES 16
void XTS_EncryptSector(
    AES_Key &k2, AES_Key &k1, u64b S, uint N,
    const u08b *pt, u08b *ct
) {
    uint i,j;
    u08b T[AES_BLK_BYTES], x[AES_BLK_BYTES];
    u08b Cin,Cout;
    assert(N >= AES_BLK_BYTES);

    for (j=0;j<AES_BLK_BYTES;j++) {
        T[j] = (u08b)(S & 0xFF);
        S = S >> 8;
    }
    AES_ECB_Encrypt(k2,T);

    for (i=0;i+AES_BLK_BYTES <= N;i+=AES_BLK_BYTES) {
        for (j=0;j<AES_BLK_BYTES;j++) x[j] = pt[i+j] ^ T[j];
        AES_ECB_Encrypt(k1,x);
        for (j=0;j<AES_BLK_BYTES;j++) ct[i+j] = x[j] ^ T[j];

        Cin=0;
        for (j=0;j<AES_BLK_BYTES;j++) {
            Cout=(T[j]>>7)&1;
            T[j]=((T[j]<<1)+Cin)&0xFF;
            Cin=Cout;
        }
        if (Cout) T[0] ^= GF_128_FDBK;
    }
}
```

```

if (i < N) {
    for (j=0; j<N; j++) {
        x[j] = pt[i+j] ^ T[j];
        ct[i+j] = ct[i+j-AES_BLK_BYTES];
    }
    for (; j<AES_BLK_BYTES; j++)
        x[j] = ct[i+j-AES_BLK_BYTES] ^ T[j];
    AES_ECB_Encrypt(k1, x);
    for (j=0; j<AES_BLK_BYTES; j++)
        ct[i+j-AES_BLK_BYTES] = x[j] ^ T[j];
}

```

학습 주석 | 핵심은 마지막 partial plaintext byte 수만큼 이전 ciphertext에서 byte를 가져오고("steal"), 남은 부분을 채워 마지막 16-byte AES 입력을 만든 뒤 다시 encrypt한다는 점입니다. 그래서 padding byte를 추가하지 않고도 ciphertext length = plaintext length를 유지합니다.

Annex D (informative) - Rationale and Design Choices

D.1 Purpose

이 Annex는 XTS-AES의 설계 선택과 그 배경 논리를 설명한다.

D.2 Transparent Encryption

이 표준의 출발점은 transform을 transparent encryption으로 사용할 수 있어야 한다는 요구사항이다. 즉 기존 data path에 encryption/decryption module을 삽입하더라도 다른 구성요소의 data layout이나 message format을 바꿀 필요가 없어야 한다.

transparent encryption은 host 내부, host와 storage device 사이 data path, storage device 내부 어디든 구현할 수 있고, data transmission protocol이나 media layout을 바꾸지 않아도 된다. sector-level storage encryption에서는 다음 두 constraint로 정리된다.

1. transform은 length-preserving이어야 한다. ciphertext 길이는 plaintext 길이와 같아야 한다. 따라서 transform은 deterministic해야 하며 ciphertext와 함께 authentication tag를 저장할 수 없다.
2. transform은 각 data unit(sector)을 다른 unit과 독립적으로, 임의의 순서로 처리할 수 있어야 한다. 이는 random access를 지원하기 위해 서로 다른 data unit 사이의 chaining을 허용하지 않는다는 뜻이다.

Working Group은 sector number에서 IV를 파생하는 CTR mode와 CBC mode를 insecure한 대안으로 보고 배제했다.

- CTR을 authentication tag 없이 사용하면 ciphertext의 특정 bit를 flip하는 것만으로 대응 plaintext bit를 flip할 수 있어 trivially malleable하다.
- CBC에서 attacker가 encrypted disk를 read/write할 수 있으면 ciphertext sector를 다른 위치로 복사하여 거의 동일한 plaintext를 얻게 할 수 있다.
- CBC에서는 이전 ciphertext block의 bit를 바꾸어 다음 plaintext block의 특정 bit를 flip할 수 있다. 대신 이전 block은 randomize되는 부작용이 있다.

XTS-AES는 ciphertext manipulation과 cut-and-paste 공격에 대해 더 나은 보호를 제공하기 때문에 선택되었다. 하지만 transparent encryption의 제약 자체 때문에 달성 가능한 security에는 본질적인 한계가 있다.

authentication tag가 없으므로 원래 ciphertext든 attacker가 수정한 ciphertext든 어떤 plaintext로는 반드시 복호화되고, alteration을 자체 검출하는 built-in mechanism이 없다. 할 수 있는 최선은 ciphertext modification이 plaintext를 완전히 randomize하게 만들고, 상위 application이 plaintext redundancy를 이용해 이런 random plaintext를 검출·폐기하도록 기대하는 것이다.

또한 transform이 deterministic이므로 같은 key와 같은 position에서 같은 plaintext를 두 번 encrypt하면 같은 ciphertext가 나온다. 서로 다른 data unit 사이 chaining이 없기 때문에 attacker는 ciphertext unit을 “mix and match”할 수 있고, 그 결과 plaintext도 같은 위치 단위로 mix and match된다.

이 weakness를 줄이기 위해 encryption/decryption에 context information을 넣을 수 있다. sector-level encryption에서 encryption과 decryption 양쪽이 공통으로 알 수 있는 context는 현재 data unit의 logical position이다. 이 position 정보를 포함하면 같은 plaintext unit이 서로 다른 위치에 기록되었음을 cryptographically 숨기고, 서로 다른 위치 사이 mix-and-match를 방지할 수 있다.

그럼에도 다음 vulnerability는 남는다.

- Traffic analysis: deterministic encryption이므로 특정 sector의 값이 이전과 달라졌는지 관찰할 수 있다.
- Replay: attacker가 특정 sector의 과거 ciphertext를 저장해 두었다가 이전 상태로 되돌릴 수 있다.
- Randomizing a sector: authentication tag가 없으므로 임의 ciphertext를 sector에 써서 application이 random plaintext를 읽게 만들 수 있다.

따라서 transparent encryption을 사용할 때는 이런 문제를 이 표준 범위 밖의 다른 mechanism으로 해결해야 한다.

D.3 Wide vs. Narrow Block Tweakable Encryption

필요한 transform interface는 다음과 같이 볼 수 있다.

$$C = \text{Enc}(K, P, i) \quad \text{and} \quad P = \text{Dec}(K, C, i)$$

여기서 P와 C는 한 sector와 같은 길이이고, K는 secret encryption key, i는 position information이다. 이상적인 security는 sector 전체 크기의 block cipher를 사용하면서 i 값마다 서로 독립적인 key를 쓰는 것처럼 보이는 것이다. Cryptographic literature에서는 이런 구조를 tweakable cipher라고 한다.

이 성질을 만족하는 wide-block construction은 존재하지만, 최소 한 sector 분량의 intermediate buffer와 sector 전체에 대한 두 번 이상의 pass가 필요한 등 비용이 크다. 더 저렴한 대안은 sector 전체 폭의 block cipher처럼 보이게 해야 한다는 요구를 완화하고, 128-bit narrow block 단위로 처리하되 서로 다른 block이 독립적인 key로 암호화된 것처럼 보이게 만드는 것이다.

서로 다른 128-bit block 사이 dependency를 포기하면 efficiency가 크게 올라간다. 대신 traffic analysis나 replay 같은 공격의 granularity가 sector 단위에서 128-bit block 단위로 더 세밀해진다. Working Group은 추가 효율성이 이 risk를 감수할 가치가 있다고 판단했다.

D.4 The XEX Construction

D.4.1 General XEX transform

[XEX04]에서 Rogaway는 AES 같은 standard cipher에서 narrow-block tweakable cipher를 만드는 construction을 설명했다. 두 key K1, K2를 사용한다. tweak는 (s,t) pair로 해석할 수 있으며 s는 sector number, t는 sector 내부 block number로 생각할 수 있다.

먼저 mask T를 계산한다.

$$T = \text{Enc}(K2, s) \otimes \alpha^t$$

여기서 multiplication은 $\text{GF}(2^n)$ 에서 수행되고 n은 underlying cipher의 block-size, α 는 $\text{GF}(2^n)$ 의 primitive element이다.

Plaintext P에서 ciphertext C를 얻는 식:

$$C = \text{Enc}(K1, P \oplus T) \oplus T$$

Ciphertext C에서 plaintext P를 얻는 식:

$$P = \text{Dec}(K1, C \oplus T) \oplus T$$

학습 주석 | 이 세 식이 XTS의 본체입니다. XTS-AES는 s에 data unit/LBA 계열 위치값을 넣고, t에 data unit 내부 block index를 반영하며, α multiplication으로 block마다 T를 바꿉니다.

D.4.2 Security of general XEX transform

[XEX04]의 generic XEX security analysis는 같은 key로 암호화하는 block 수가 underlying block cipher의 birthday bound $2^{n/2}$ 보다 충분히 작을 때 secure함을 보인다. block 수가 $2^{n/2}$ 에 접근하면 일부 attack이 가능해진다.

분석상의 attacker는 arbitrary plaintext/ciphertext와 arbitrary tweak value로 encryption/decryption query를 할 수 있다. 응답은 XEX construction 또는 tweak마다 서로 독립인 truly random permutation collection 중 하나에서 나온다. attacker의 목표는 어느 경우인지 구별하는 것이다.

Rogaway는 q 개의 query를 하는 attacker의 distinguishing advantage가 random guess 대비 다음보다 크지 않음을 보였다.

$$4.5 q^2 / 2^n + \epsilon$$

ϵ 는 q query에서 underlying cipher와 random permutation을 구별하는 advantage를 나타내는 error term이다.

이 분석은 real-world attacker가 더 많은 정보를 갖기 어렵다는 점에서 실제 XTS-AES 사용과 관련된다. 즉 XEX를 random permutation collection과 유의미하게 구별할 수 없다면, 실제 attack은 각 128-bit block마다 독립 random permutation을 사용했을 때에도 비슷하게 성공할 수 있어야 한다.

따라서 attacker가 얻는 핵심 정보는 다음 정도다.

- 같은 plaintext와 같은 tweak는 항상 같은 ciphertext가 된다.
- 같은 ciphertext와 같은 tweak는 항상 같은 plaintext로 복호화된다.
- 그 외 다른 ciphertext/plaintext는 거의 random value로 decrypt/encrypt된다.

즉 error term을 제외하면 XEX에 대한 공격은 n -bit granularity의 transparent encryption이 본질적으로 갖는 attack에 해당한다.

q 가 birthday bound에 접근하면 collision을 이용한 attack이 가능하다. Known-plaintext attacker가 q 개의 $((s_i, t_i), P_i, C_i)$ tuple을 본다고 하자. 각 tweak에서 계산되는 mask를 T_i 라고 한다. $q \approx 2^{n/2}$ 이면 다음 collision이 발생할 확률이 무시할 수 없게 된다.

$$P_i \oplus T_i = P_j \oplus T_j$$

이 경우 다음도 성립한다.

$$C_i \oplus T_i = \text{Enc}(K1, P_i \oplus T_i) = \text{Enc}(K1, P_j \oplus T_j) = C_j \oplus T_j$$

두 equality를 XOR하면 다음을 얻는다.

$$P_i \oplus C_i = P_j \oplus C_j$$

attacker는 모든 i 에 대해 $S_i = P_i \oplus C_i$ 를 계산하고 $S_i = S_j$ 인 pair 수를 세어 XEX를 random permutation collection과 구별하는 데 사용할 수 있다.

collision이 확인되었다고 가정하면 attacker가 position i 에서 정상적으로 encrypted data를 만들 수 있고 position j 의 ciphertext를 수정할 수 있을 때, j 의 plaintext를 원하는 방향으로 수정하는 방법도 있다.

$\Delta \neq 0$ 에 대해 position i 에서 $P'_i = P_i \oplus \Delta$ 를 encrypt하고 corresponding ciphertext C'_i 를 얻는다. 그리고 position j 의 ciphertext를 다음으로 바꾼다.

$$C'_j = C_j \oplus (C_i \oplus C'_i)$$

그러면 새 ciphertext block은 $P'_j = P_j \oplus \Delta$ 로 복호화된다. 원문 전개는 다음 관계를 사용한다.

$$C'_j \oplus T_j = C'_i \oplus T_i$$

따라서

$$\text{Dec}(K1, C'_j \oplus T_j) = \text{Dec}(K1, C'_i \oplus T_i)$$

이고 최종적으로

$$P'_j = P_j \oplus \Delta$$

가 된다.

D.4.3 XTS-AES as a specific instantiation of general XEX

이 표준의 XTS-AES-128과 XTS-AES-256은 AES를 underlying block cipher로 쓰는 XEX의 구체적 instance이며 $n=128$ 이다. data unit sequence number, 즉 상대적 위치를 tweak로 사용하여 key scope 또는 일부 key scope를 재암호화 없이 copy/backup할 수 있게 한다.

Generic XEX와 달리 이 표준의 XTS-AES는 tweaking 용도와 data encryption 용도에 서로 다른 key를 사용한다. 목적별 key separation은 좋은 security design practice로 간주된다.

4.5 $q^2/2^n$ 은 q 가 2^{40} 보다 크게 증가하지 않는 동안 충분히 작다. [XEX04] proof는 같은 key로 약 1 TB 정도($q \approx 2^{36}$ block)를 encrypt하는 경우 강한 security guarantee를 준다. 이 경우 attack success probability는 약 2^{-53} 보다 좋지 않다.

같은 key로 더 많은 데이터를 암호화하면 guarantee는 약해진다. 원문은 petabyte 규모에서 약 2^{-37} , exabyte 규모에서 약 2^{-17} 수준의 upper bound example을 제시한다.

따라서 하나의 key로 암호화할 최대 data amount를 정할 때는 이러한 계산과 실제 attack implication을 함께 고려해야 한다.

D.5 Sector-size that is not a multiple of 128 bits

Generic XEX는 128-bit block의 정수배 sector에는 즉시 적용할 수 있다. sector 내부 block number를 tweak 일부로 사용해 각 128-bit block에 transform을 독립 적용하면 된다. 512 byte, 4096 byte 같은 일반 sector size가 여기에 해당한다.

하지만 520-byte sector처럼 128 bit의 정수배가 아닌 sector에는 직접 적용되지 않는다. 이런 경우 표준은 ECB mode의 방식과 유사한 ciphertext stealing을 사용한다. 마지막 full block을 제외한 full block들을 각각 다른 tweak로 encrypt하고, 마지막 full block과 남은 partial block은 5.3.2의 두 번의 XTS-AES-blockEnc를 사용해 함께 처리한다.

D.6 Miscellaneous - 일반 사용상 주의

- XTS-AES-[128,256] secret key는 random 또는 cryptographically strong pseudo-random source에서 생성해야 한다. Low-entropy source에서 key를 고르면 XEX04의 security guarantee는 무효가 된다.
- 하나의 cryptographic key로 수백 TB를 크게 넘는 데이터를 encrypt하면 D.4.3에서 설명한 attack 가능성이 커진다. 이 limitation은 XTS만의 특성이 아니라 AES block size가 128 bit라는 점에서 직접 나온다. AES-256 key를 사용해도 AES block size는 128 bit이므로 이 bound 자체가 사라지는 것은 아니다.

학습 주석 | 이 2007 Draft의 수취·권고는 당시 문서의 설명입니다. 현재 실제 설계에서는 최신 IEEE 1619 / NIST SP 800-38E 및 제품의 security requirement를 함께 확인해야 합니다. 이 번역본에서는 원문의 논리를 그대로 보존했습니다.